

龙鱼MobileSDK功能说明

一、文档变更记录

版本	变更说明	日期	作者	审核
V1.0	创建文档	2024/9/26	冯银华	

二、概述

本教程旨在帮助您基本了解龙鱼MobileSDK 的相关功能以及相关接口的使用。DFMobileSDK包含了与 飞机各个模块通信的服务，主要包含通用模块、飞行任务模块、相机模块、飞行控制模块、飞行参数模块、云台模块、视觉模块以及图传模块，同时也提供飞机连接状态的监听以及遥控器模块的相关功能。下面为DFMobileSDK讲解如何在开发中使用这些功能

适用范围：目前仅仅支持Autel 龙鱼9.7寸平板上开发

三、环境配置及接入

1、Android Studio 配置工程（目前暂时不支持Eclipse）

2、注册成为开发者

在 [Autel开发者平台](#) 注册成为开发者，创建应用，为创建的应用申请唯一APP KEY，初始化SDK需要用到。一个应用对应一个APP KEY。

3、Sample Demo示例

开发者可以通过sdk示例代码，快速了解到SDK所提供的API接口。

[Android 示例工程代码](#) 使用分支为feature/V3 表示为支持新航线功能

4、 创建一个Android工程，导入aar包，在开发模块的gradle文件中添加依赖

```
1 repositories {
2     flatDir {
3         dirs 'libs'
4     }
5 }
```

```

1
2  api(name: "autel-sdk-release_V2.0.16", ext: "aar") {
3      exclude module: 'okio'
4      exclude module: 'okhttp'
5  }

```

NOTE: 开发者可根据自己的实际情况修改SDK 本地依赖路径

5、基础权限申请，在AndroidManifest.xml文件中添加以下元素

```

1
2 <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
3 <uses-permission android:name="android.permission.INTERNET" />
4 <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
5 <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
6 <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
7 <uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
8 <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
9 <uses-permission android:name="android.permission.RECORD_AUDIO" />

```

6、初始化SDK，在需要使用SDK功能前，在Application中调用init函数初始化相关SDK功能

自定义Application，在Application中添加SDK 的初始化

```

1 /**
2  * 初始化SDK，通过网络验证APPKey的有效性
3  */
4  String appKey = "<SDK license should be input>";
5  AutelSdkConfig config = new AutelSdkConfig.AutelSdkConfigBuilder()
6      .setAppKey(appKey)
7      .setPostOnUi(true)
8      .create();
9  Autel.init(this, config, new CallbackWithNoParam() {
10      @Override
11      public void onSuccess() {
12          Log.v(TAG, "checkAppKeyValidate onSuccess");
13      }
14
15      @Override

```

```

16     public void onFailure(AutelError error) {
17         Log.v(TAG, "checkAppKeyValidate " + error.getDescription());
18     }
19 });
20 //创建日志存储目录
21 File myDir = new File(AutelDirPathUtils.getLogCatPath());
22 if (!myDir.exists()) {
23     myDir.mkdirs();
24 }
25 //使用基站连接时设置0，使用图传直连时设置为1，图传直连距离在1km以内
26 NetworkProxyJni.setType(0);

```

说明：Autel.init()函数的第一个参数是会被长期持有的Context对象，为避免内存泄露，建议使用Application的Context对象

NOTE: 在整个应用结束Autel SDK服务的使用后（一般在退出APP时），调用 Autel.destroy()来结束Autel SDK相关资源的调用

NOTE: appKey 为开发者在Autel开发者平台申请的应用关联钥匙

7、SDK Api接口介绍

SDK提供以下模块的功能服务：Album（相册）、Battery（电池）、Camera（相机）、DSP（图传）、FlyController（飞行控制器）、Gimbal（云台）、Mission（任务）、RemoteController（遥控器）、Codec(视频解码)

开发者通过与产品连接的监听回调，获取到BaseProduct接口，从而获取相关模块的接口（以下为各个模块的接口获取，详细的各个接口用法可以参考对应模块提供的api文档）：

```

1 //设置飞机设备连接监听
2 Autel.setProductConnectListener(
3     new ProductConnectListener() {
4         @Override
5         public void productConnected(
6             BaseProduct product) {
7             //.....
8             /**
9              * 获取到的product，即飞行器对外提供的各个模块的接口方法
10             */
11             //.....
12         }
13     }

```

获取飞行器对外提供的接口，从而可以获取相关产品的模块接口，例如：

```

1  switch (product.getType()){
2      //对应龙鱼的不同型号
3      case DRAGONFISH:
4          break;
5      case DRAGONFISH_7_5_VTOL:
6          break;
7      case DRAGONFISH_15_VTOL:
8          break;
9      case DRAGONFISH_30_VTOL:
10         break;
11 }
12 //相机
13 AutelCameraManager autelCameraManager = baseProduct.getCameraManager()
14 //相册
15 AutelAlbum album = product.getAlbum();
16 //DSP图传
17 CruiserDsp cruiserDsp = (CruiserDsp) product.getDsp();
18 //飞控
19 CruiserFlyController cruiserFlyController = (CruiserFlyController)
    product.getFlyController();
20 //电池
21 CruiserBattery cruiserVisual = (CruiserBattery) product.getBattery();
22 //航线任务
23 CruiserMissionManager missionManager = (CruiserMissionManager)
    product.getMissionManager();
24 //云台
25 CruiserGimbal gimbal = (CruiserGimbal) product.getGimbal();

```

```

1  //设置相机连接监听，代码参考Sample Demo里面CameraActivity中
2  autelCameraManager.setCameraChangeListener(new
    CallbackWithTwoParams<CameraProduct, AutelBaseCamera>() {
3      @Override
4      public void onSuccess(final CameraProduct data1, final AutelBaseCamera
        data2) {
5          Log.v(TAG, "initListener onSuccess connect " + data1);
6          if (currentCamera == data2) {
7              return;
8          }
9          //相机接口对象
10         currentCamera = data2;
11         cameraType.setText(data1.toString());
12         //根据不同的相机类型，切换不同的页面
13         switch (data1) {
14             case XT708://双光相机
15             case XT710:

```

```

16         case XT711:
17         case XT713:
18         case XT714:
19         case XT715:
20         case XT717:
21             changePage(CameraDFFragment.class);
22             break;
23
24         default:
25             changePage(CameraNotConnectFragment.class);
26     }
27
28 }
29
30 @Override
31 public void onFailure(AutelError error) {
32     Log.v(TAG, "initListener onFailure error " + error.getDescription());
33     cameraType.setText("currentCamera connect broken " +
34         error.getDescription());
35 }

```

通过接口回调获取到BaseProduct，就可以通过BaseProduct去后去飞行器的各个模块，详情可参考各模块接口文档说明

四、模块接口Api说明

1、相机功能实现教程

说明：相机接口类获取，以demo里面的CameraActivity为例：

1.1相机基础接口类AutelBaseCamera以及方法介绍

```

1 /**
2  * Sets the listener for the current media mode to monitor the real-time data.
3  * @param callback the listener will be cancelled when the callback is null.
4  */
5 void setMediaModeListener(CallbackWithOneParam<MediaMode> callback);
6
7 /**
8  * Sets the listener for the SD card to monitor the real-time data.
9  * @param callback the listener will be cancelled when the callback is null.
10 */
11 void setSDCardStateListener(CallbackWithOneParam<SDCardState> callback);

```

```
12
13
14 void setMMCCardStateListener(CallbackWithOneParam<MMCState> callback);
15 /**
16  * Sets the listener for the camera shooting and recording to monitor the real-
17  * time data.
18  * @param callback the listener will be cancelled when the callback is null.
19  */
20 void setMediaStateListener(CallbackWithTwoParams<MediaStatus, String>
21 callback);
22 /**
23  * Sends the format SD card command to the camera server.
24  * <p>
25  * Note: first, see if the camera is in idle condition. If yes, then this
26  * operation can be continued.
27  * @param callback the callback when format the SD card.
28  */
29 void formatSDCard(CallbackWithNoParam callback);
30 /**
31  * Sends the camera reset command to the camera server.
32  * <p>
33  * Note: first, see if the camera is in idle condition. If yes, then this
34  * operation can be continued.
35  * @param callback the callback when reset the camera.
36  */
37 void resetDefaults(CallbackWithNoParam callback);
38 /**
39  * Returns the current camera working state.
40  * @param callback the callback when get the current camera working state.
41  */
42 void getWorkState(CallbackWithOneParam<WorkState> callback);
43 /**
44  * Returns the current SD card state.
45  * @param callback the callback when get the current SD card state.
46  */
47 void getSDCardState(CallbackWithOneParam<SDCardState> callback);
48 /**
49  * Returns the free space of the SD card in MB.
50  * @param callback the callback when get the free space of the SD card.
51  */
52 void getSDCardFreeSpace(CallbackWithOneParam<Long> callback);
```

```
55
56 /**
57  * Gets the camera model.
58  * @return the camera model.
59  */
60 CameraProduct getProduct();
61
62 /**
63  * Returns the current version of the camera.
64  * @param callback the callback when get the current version of the camera.
65  */
66 void getVersion(CallbackWithOneParam<String> callback);
67
68 /**
69  * Sets the camera media mode.
70  * @param mediaMode the media mode enum.
71  * @param callback the callback when set the camera media mode.
72  */
73 void setMediaMode(MediaMode mediaMode, CallbackWithNoParam callback);
74
75 /**
76  * Returns the current camera media mode.
77  * @param callback the callback when get the current camera media mode.
78  */
79 void getMediaMode(CallbackWithOneParam<MediaMode> callback);
80
81 /**
82  * Request the camera to take a photo.
83  * <p>
84  * Note:
85  * </p>
86  * <p>
87  * 1. Firstly, see if the camera is in the photo shooting mode.
88  * </p>
89  * <p>
90  * 2. See if the camera is idle in the current mode.
91  * </p>
92  * <p>
93  * 3. See if the SD card is in the available state (CARD_READY, LOW_SPEED_CARD
    are all available).
94  * </p>
95  * @param callback the callback when start taking photos.
96  */
97 void startTakePhoto(CallbackWithNoParam callback);
98
99 /**
100  * Request the camera to start recording.
```

```
101 * <p>
102 * Note:
103 * </p>
104 * <p>
105 * 1. See if the camera is in the recording mode.
106 * </p>
107 * <p>
108 * 2. See if the camera is idle in the current mode.
109 * </p>
110 * <p>
111 * 3. See if SD card is in the available state.
112 * </p>
113 * @param callback the callback when start recording videos.
114 */
115 void startRecordVideo(CallbackWithNoParam callback);
116
117 /**
118 * Request the camera to stop recording.
119 * <p/>
120 * Note:
121 * <p/>
122 * See if the camera is recording now.
123 * @param callback the callback when stop recording videos.
124 */
125 void stopRecordVideo(CallbackWithNoParam callback);
126
127 /**
128 * Request the camera to stop taking time lapse photos.
129 * <p/>
130 * Note:
131 * <p/>
132 * See if the camera is taking time lapse photos now.
133 * @param callback the callback when stop taking time lapse photos.
134 */
135 void stopTakePhoto(CallbackWithNoParam callback);
136
137 /**
138 * Returns the current recording time.
139 * @param callback the callback when get the current recording time.
140 */
141 void getCurrentRecordTime(CallbackWithOneParam<Integer> callback);
142
143 /**
144 * Returns the state information of the camera (returns R12StateInfo for R12,
    returns XB015StateInfo for XB015).
145 * @param callback the callback when get the state information of the camera.
146 */
```



```

147 void getStateInfo(CallbackWithOneParam<BaseStateInfo> callback);
148
149 /**
150  * Converts to ReactiveX interface.
151  * @return ReactiveX interface.
152  */
153 RxAutelBaseCamera toRx();
154
155 /**
156  * 销毁监听
157  * @param callback
158  */
159 void setDataDestructionListener(final CallbackWithOneParam<DataDestruction>
callback);

```

1.2 龙鱼系列相机AutelMultiCamera接口抽象类介绍

```

1 /**
2  * 设置联动变焦相机状态上报
3  * @param callback
4  */
5 void setCameraLinkFovStatusListener(CallbackWithOneParam<CameraLinkStatus>
callback);
6
7 /**
8  * 设置温度告警状态监听上报
9  * @param callback
10 */
11 void
setTemperatureWarningListener(CallbackWithOneParam<IrTempetureWarningParams>
callback);
12
13 /**
14  * Sets the listener for the camera information to monitor the real-time data.
15  *
16  * @param callback the listener will be cancelled when the callback is null.
17  */
18 void setInfoListener(CallbackWithOneParam<CameraInfo> callback);
19
20 /**
21  * Sets the listener for the camera auto focus to monitor the real-time data.
22  *
23  * @param callback the listener will be cancelled when the callback is null.
24  */

```

```

25 void setAutoFocusStateListener(CallbackWithTwoParams<LensFocusStatus,
    List<SpotMeteringArea>> callback);
26
27 /**
28  * Set AF center status monitoring
29  *
30  * @param callback the listener will be cancelled when the callback is null.
31  */
32 void setAFCenterListener(CallbackWithOneParam<Boolean> callback);
33
34 /**
35  * 待翻译 Setting中某一个选项发生变化时触发
36  * Triggered when any setting data changes
37  *
38  * @param callback
39  */
40 void setSettingChangeListener(CallbackWithOneParam<SettingEvent> callback);
41
42 /**
43  * Sets the listener for the camera histogram to monitor the real-time data.
44  *
45  * @param callback the listener will be cancelled when the callback is null.
46  */
47 void setHistogramListener(CallbackWithOneParam<int[]> callback);
48
49
50 /**
51  * Sets the auto exposure lock for XB015 camera.
52  * <p/>
53  * Note:
54  * <p/>
55  * The auto exposure lock will be set as DISABLE in manual mode.
56  * <p/>
57  * Only LOCK and UNLOCK are supported for the auto exposure lock in other
    modes.
58  *
59  * @param lockState the enum values of the auto exposure lock.
60  * @param callback the callback when set the auto exposure lock for XB015
    camera.
61  */
62 void setAutoExposureLockState(AutoExposureLockState lockState,
    CallbackWithNoParam callback);
63
64 /**
65  * Returns the auto exposure lock state.
66  *
67  * @param callback the callback when get the auto exposure lock state.

```

```

68 */
69 void getAutoExposureLockState(CallbackWithOneParam<AutoExposureLockState>
    callback);
70
71 /**
72  * Sets the camera exposure values.
73  * <p/>
74  * Note: This function is only available in auto mode.
75  *
76  * @param exposure the camera exposure values.
77  * @param callback the callback when set the camera exposure values.
78  */
79 void setExposure(ExposureCompensation exposure, CallbackWithNoParam callback);
80
81 /**
82  * Returns the camera exposure values.
83  *
84  * @param callback the callback when get the camera exposure values.
85  */
86 void getExposure(CallbackWithOneParam<ExposureCompensation> callback);
87
88 /**
89  * Sets the camera ISO values.
90  * <p/>
91  * Note: This function is only available in manual mode.
92  *
93  * @param value the camera ISO values.
94  * @param callback the callback when set the camera ISO values.
95  */
96 void setISO(CameraISO value, CallbackWithNoParam callback);
97
98 /**
99  * Returns the camera ISO values.
100  *
101  * @param callback the callback when get the camera ISO values.
102  */
103 void getISO(CallbackWithOneParam<CameraISO> callback);
104
105 /**
106  * Sets the camera shutter values.
107  * <p/>
108  * Note: This function is only available in manual mode.
109  *
110  * @param value the camera shutter values.
111  * @param callback the callback when set the camera shutter values.
112  */
113 void setShutter(ShutterSpeed value, CallbackWithNoParam callback);

```

```

114
115 /**
116  * Returns the shutter setting values.
117  *
118  * @param callback the callback when get the shutter setting values.
119  */
120 void getShutter(CallbackWithOneParam<ShutterSpeed> callback);
121
122 /**
123  * Sets the camera color styles.
124  *
125  * @param value the enum of the camera color styles.
126  * @param callback the callback when set the camera color styles.
127  */
128 void setColorStyle(ColorStyle value, CallbackWithNoParam callback);
129
130 /**
131  * Returns the camera color styles.
132  *
133  * @param callback the callback when get the camera color styles.
134  */
135 void getColorStyle(CallbackWithOneParam<ColorStyle> callback);
136
137 /**
138  * Sets the camera white balance values.
139  *
140  * @param value the enum of the camera white balance values.
141  * @param callback the callback when set the camera white balance values.
142  */
143 void setWhiteBalance(WhiteBalance value, CallbackWithNoParam callback);
144
145 /**
146  * Returns the camera white balance values.
147  *
148  * @param callback the callback when get the camera white balance values.
149  */
150 void getWhiteBalance(CallbackWithOneParam<WhiteBalance> callback);
151
152
153 /**
154  * Sets the exposure mode for XB015 camera.
155  *
156  * @param param the enum values of the camera exposure mode.
157  * @param callback the callback when set the exposure mode for XB015 camera.
158  */
159 void setExposureMode(ExposureMode param, CallbackWithNoParam callback);
160

```

```

161 /**
162  * Returns the exposure mode of XB015 camera.
163  *
164  * @param callback the callback when get the exposure mode of XB015 camera.
165  */
166 void getExposureMode(CallbackWithOneParam<ExposureMode> callback);
167
168 /**
169  * Sets the photo style for XB015 camera.
170  *
171  * @param value the enum values of the photo style.
172  * @param callback the callback when set the photo style for XB015 camera.
173  */
174 void setPhotoStyle(PhotoStyleType value, CallbackWithNoParam callback);
175
176 /**
177  * Sets the custom values for the photo style.
178  *
179  * @param contrast the contrast value, ranges from -3 to 3.
180  * @param saturation the saturation value, ranges from -3 to 3.
181  * @param sharpness the sharpness value, ranges from -3 to 3.
182  * @param callback the callback when set the custom values for the photo
183  style.
184  */
185 void setPhotoStyle(int contrast, int saturation, int sharpness,
186 CallbackWithNoParam callback);
187
188 /**
189  * Returns the photo style of XB015 camera.
190  *
191  * @param callback the callback when get the photo style of XB015 camera.
192  */
193 void getPhotoStyle(CallbackWithOneParam<PhotoStyle> callback);
194
195 /**
196  * Set camera focus mode
197  *
198  * @param param param CameraLensFocusMode enum value to be set
199  */
200 void setFocusMode(LensFocusMode param, CallbackWithNoParam callback);
201
202 /**
203  * Obtain focus mode
204  *
205  * @return

```

```

206 */
207 void getFocusMode(CallbackWithOneParam<LensFocusMode> callback);
208
209 /**
210  * @return custom color temperature of white balance
211  * Range from 2000 to 10000,100 per time
212  */
213 void getColorTemperature(CallbackWithOneParam<Integer> callback);
214
215 /**
216  * Sets the switch values for the video subtitle.
217  *
218  * @param enable whether the video subtitle switch is enabled or not.
219  * @param callback the callback when set the switch values for the video
  subtitle.
220  */
221 void setVideoSubtitleEnable(boolean enable, CallbackWithNoParam callback);
222
223 /**
224  * Returns the switch values to get the video subtitle switch status.
225  *
226  * @param callback the callback of turning on/off the video subtitle switch.
227  */
228 void isSubtitleEnable(CallbackWithOneParam<Boolean> callback);
229
230 /**
231  * Sets the digital zoom scale, ranges from 100 to 800.
232  *
233  * @param factor the integer between 100 and 800.
234  * @param callback the callback when set the digital zoom scale.
235  */
236 void setDigitalZoomScale(int factor, CallbackWithNoParam callback);
237
238 /**
239  * 设置数码变焦倍数
240  * @param factor 变焦倍数
241  * @param mainScreenCamNo 主屏相机编号
242  * @param callback 回调
243  */
244 void setDigitalZoomScale(int factor, int mainScreenCamNo, CallbackWithNoParam
  callback);
245
246 /**
247  * 设置数码变焦倍数 用于博东波轮后持续变焦功能
248  * @param cmd 1:放大 2:缩小 0:停止
249  * @param callback 回调
250  */

```

```
251 void setDigitalZoomScaleRollStep(int cmd, CallbackWithNoParam callback);
252
253 /**
254  * Returns the digital zoom scale.
255  *
256  * @param callback the callback when get the digital zoom scale.
257  */
258 void getDigitalZoomScale(CallbackWithOneParam<Integer> callback);
259
260 /**
261  * Sets different photo counts for each shooting in burst mode.
262  *
263  * @param value the enum values of the photo count in burst mode.
264  * @param callback the callback when set different photo counts for each
265  * shooting in burst mode.
266  */
267 void setPhotoBurstCount(PhotoBurstCount value, CallbackWithNoParam callback);
268
269 /**
270  * Returns different photo counts of each shooting in burst mode.
271  *
272  * @param callback the callback when get different photo counts of each
273  * shooting in burst mode.
274  */
275 void getPhotoBurstCount(CallbackWithOneParam<PhotoBurstCount> callback);
276
277 /**
278  * Sets the time interval values between taking two photos in timelapse mode.
279  *
280  * @param value the enum values of the time interval between taking two
281  * photos.
282  * @param callback the callback when set the time interval values between
283  * taking two photos in timelapse mode.
284  */
285 void setPhotoTimelapseInterval(PhotoTimelapseInterval value,
286 CallbackWithNoParam callback);
287
288 /**
289  * Returns the time interval values of timelapse mode.
290  *
291  * @param callback the callback when get the time interval values of timelapse
292  * mode.
293  */
294 void getPhotoTimelapseInterval(CallbackWithOneParam<PhotoTimelapseInterval>
295 callback);
296
297 /**
```

```
291 * Sets different photo counts for each shooting in AEB mode.
292 *
293 * @param value the enum values of the photo count in AEB mode.
294 * @param callback the callback when set different photo counts for each
    shooting in AEB mode.
295 */
296 void setPhotoAEBCount(PhotoAEBCount value, CallbackWithNoParam callback);
297
298 /**
299 * Returns different photo counts of each shooting in AEB mode.
300 *
301 * @param callback the callback when get different photo counts of each
    shooting in AEB mode.
302 */
303 void getPhotoAEBCount(CallbackWithOneParam<PhotoAEBCount> callback);
304
305 /**
306 * Returns the number of videos.
307 *
308 * @param callback the callback when get the number of videos.
309 */
310 void getVideoSum(CallbackWithOneParam<VideoSum> callback);
311
312 /**
313 * Returns the number of photos can be taken.
314 *
315 * @param callback the callback when get the number of photos can be taken.
316 */
317 void getLeftPhotoSum(CallbackWithOneParam<Integer> callback);
318
319 /**
320 * Sets the video format for XB015 camera.
321 *
322 * @param value the enum values of the video format.
323 * @param callback the callback when set the video format for XB015 camera.
324 */
325 void setVideoFormat(VideoFormat value, CallbackWithNoParam callback);
326
327 /**
328 * Returns the video format of XB015 camera.
329 *
330 * @param callback the callback when get the video format of XB015 camera.
331 */
332 void getVideoFormat(CallbackWithOneParam<VideoFormat> callback);
333
334 /**
335 * Sets the video standard for XB015 camera.
```



```

336 *
337 * @param value the enum values of the video standard.
338 * @param callback the callback when set the video standard for XB015 camera.
339 */
340 void setVideoStandard(VideoStandard value, CallbackWithNoParam callback);
341
342 /**
343 * Returns the video standard of XB015 camera.
344 *
345 * @param callback the callback when get the video standard of XB015 camera.
346 */
347 void getVideoStandard(CallbackWithOneParam<VideoStandard> callback);
348
349 /**
350 * Sets the photo format for XB015 camera.
351 *
352 * @param value the enum values of the photo format.
353 * @param callback the callback when set the photo format for XB015 camera.
354 */
355 void setPhotoFormat(PhotoFormat value, CallbackWithNoParam callback);
356
357 /**
358 * Returns the photo format of XB015 camera.
359 *
360 * @param callback the callback when get the photo format of XB015 camera.
361 */
362 void getPhotoFormat(CallbackWithOneParam<PhotoFormat> callback);
363
364 /**
365 * Sets the photo size for XB015 camera.
366 *
367 * @param value the enum values of the photo size.
368 * @param callback the callback when set the photo size for XB015 camera.
369 */
370 void setAspectRatio(PhotoAspectRatio value, CallbackWithNoParam callback);
371
372 /**
373 * Returns the current photo size.
374 *
375 * @param callback the callback when get the current photo size.
376 */
377 void getAspectRatio(CallbackWithOneParam<PhotoAspectRatio> callback);
378
379 /**
380 * Sets the video resolution and the associated frame rate.
381 * <p>

```

```

382 * Note: See the current standard first. NTSC can not be set the resolution
    for PAL, and vice versa.
383 *
384 * @param value the enum values of the video resolution and the associated
    frame rate.
385 * @param callback the callback when set the video resolution and the
    associated frame rate.
386 */
387 void setVideoResolutionAndFrameRate(VideoResolutionAndFps value,
    CallbackWithNoParam callback);
388
389 /**
390 * Returns the current video resolution and the associated frame rate.
391 *
392 * @param callback the callback when get the video resolution and the
    associated frame rate.
393 */
394 void
    getVideoResolutionAndFrameRate(CallbackWithOneParam<VideoResolutionAndFps>
    callback);
395
396 /**
397 * Sets the video encoding format.
398 *
399 * @param encoding the enum values of the video encoding format.
400 * @param callback the callback when set the video encoding format.
401 */
402 void setVideoEncodeFormat(VideoEncodeFormat encoding, CallbackWithNoParam
    callback);
403
404 /**
405 * Returns the video encoding format.
406 *
407 * @param callback the callback when get the video encoding format.
408 */
409 void getVideoEncodeFormat(CallbackWithOneParam<VideoEncodeFormat> callback);
410
411 /**
412 * Sets the rotation for the camera video.
413 *
414 * @param mode the enum values of the video rotation.
415 * @param callback the callback when set the rotation for the camera video.
416 */
417 void setVideoRotation(VideoRotation mode, CallbackWithNoParam callback);
418
419 /**
420 * Returns the rotation of the camera video.

```

```

421  *
422  * @param callback the callback when get the rotation of the camera video.
423  */
424  void getVideoRotation(CallbackWithOneParam<VideoRotation> callback);
425
426
427  /**
428  * Sets the PIV mode for XB015 camera.
429  *
430  * @param mode the enum values of the PIV mode.
431  * @param callback the callback when set the PIV mode for XB015 camera.
432  */
433  void setPIVMode(PIVMode mode, CallbackWithNoParam callback);
434
435  /**
436  * Returns the PIV mode of XB015 camera.
437  *
438  * @param callback the callback when get XB015 camera.
439  */
440  void getPIVMode(CallbackWithOneParam<PIVMode> callback);
441
442  /**
443  * Sets the timelapse interval for the auto PIV mode.
444  *
445  * @param interval the enum values of the timelapse interval.
446  * @param callback the callback when set the timelapse interval for the auto
PIV mode.
447  */
448  void setAutoPIVTimelapseInterval(VideoSnapshotTimelapseInterval interval,
CallbackWithNoParam callback);
449
450  /**
451  * Returns the timelapse interval of the auto PIV mode.
452  *
453  * @param callback the callback when get the timelapse interval of the auto
PIV mode.
454  */
455  void getAutoPIVTimelapseInterval(CallbackWithOneParam<VideoSnapshotTimelapseInterval>
  > callback);
456
457  /**
458  * Switches the camera from the record mode to the previous shoot mode.
459  *
460  * @param callback the callback when switch the camera from the record mode to
the previous shoot mode.
461  */

```

```

462 void switchToPreviousPhotoMode(CallbackWithOneParam<MediaMode> callback);
463
464 /**
465  * Set camera aperture value
466  * <p/>
467  * Note:
468  * <p/>
469  * 1. This is only available in Manual and AperturePriority modes.
470  *
471  * @param value aperture value to be set
472  */
473 void setCameraAperture(CameraAperture value, CallbackWithNoParam callback);
474
475 /**
476  * 获取相机光圈值
477  */
478 void getCameraAperture(CallbackWithOneParam<CameraAperture> callback);
479
480 /**
481  * 设置相机照片质量
482  * @param level 照片质量
483  * @param callback
484  */
485 void setPhotoQuality(PhotoQualityLevel level, CallbackWithNoParam callback);
486
487 /**
488  * 获取相机照片质量
489  * @param callback
490  */
491 void getPhotoQuality(CallbackWithOneParam<PhotoQualityLevel> callback);
492
493 /**
494  * 设置相机拍照后照片存储位置
495  * @param saveLocation
496  * @param callback
497  */
498 void setAlbumSaveLocation(final SaveLocation saveLocation, CallbackWithNoParam
callback);
499
500 /**
501  * 获取相机拍照后照片存储位置
502  * @param callback
503  */
504 void getAlbumLocation(CallbackWithOneParam<SaveLocation> callback);
505
506 /**
507  * 格式化闪存卡

```

```

508 * @param callback
509 */
510 void formatFlashMemoryCard(CallbackWithNoParam callback);
511
512 /**
513 * 设置MF聚焦测光点
514 *
515 * @param x raster No.x in horizontal
516 * @param y raster No.y in vertical
517 * @param iCompletionCallbackWith
518 */
519 void setFocusMFSpotArea(int x, int y, final CallbackWithNoParam
iCompletionCallbackWith);
520
521 /**
522 * 获取MF聚焦测光点
523 *
524 * @param callbackWithOneParam
525 */
526 void getFocusMFSpotArea(CallbackWithOneParam<SpotMeteringArea>
callbackWithOneParam);
527
528 /**
529 * Get camera focus distance
530 *
531 * @param callback the callback when get camera focus distance.
532 */
533 void getFocusDistance(CallbackWithOneParam<Integer> callback);
534
535 /**
536 * <p/>
537 * Set camera focus position, , ranges from 10 cm to 10000 cm (Only valid MF
mode).
538 * <p/>
539 * Unit: cm
540 * <p/>
541 * Note: this is available in CameraLensFocusMode MF mode.
542 */
543 void setFocusDistance(int objectDistance, CallbackWithNoParam callback);
544
545 /**
546 * Set camera Auto Focus meter
547 *
548 * @param x raster No.x in horizontal lines, ranges from 1 to 100
549 * @param y raster No.y in vertical lines, ranges from 1 to 100
550 * Note: this is available in AF mode
551 */

```

```
552 void setPhotoAutoFocusMeter(int x, int y, CallbackWithNoParam callback);
553
554 /**
555  * 拍照时云台锁定功能
556  *
557  * @param state
558  * @param callback
559  */
560 void setGimbalLockState(AutelSwitchState state, CallbackWithNoParam callback);
561
562 /**
563  * 拍照时云台锁定功能
564  *
565  * @param callback
566  */
567 void getGimbalLockState(CallbackWithOneParam<AutelSwitchState> callback);
568
569 /**
570  * Sets the switch values for the 3d noise reduction.
571  *
572  * @param enable whether enable the 3d noise reduction or not.
573  * @param level set 3d noise reduction level.
574  * @param callback the callback when set the switch values for the 3d noise
575  * reduction.
576  */
577 void set3DNoiseReduction(boolean enable, int level, CallbackWithNoParam
578 callback);
579
580 /**
581  * get 3d noise reduction params.
582  */
583 void get3DNoiseReduction(CallbackWithTwoParams<Boolean, Integer> callback);
584
585 /**
586  * 设置直方图开头
587  *
588  * @param enable
589  * @param callback
590  */
591 void setHistogramEnable(boolean enable, CallbackWithNoParam callback);
592
593 /**
594  * MF辅助对焦使能
595  *
596  * @param enable
597  * @param callback
598  */
```

```

597 void setMFAssistFocusEnable(boolean enable, CallbackWithNoParam callback);
598
599 /**
600  * MF辅助对焦使能
601  *
602  * @param callback
603  */
604 void getMFAssistFocusEnable(CallbackWithOneParam<Boolean> callback);
605
606 /**
607  * 设置雾透使能开关
608  * @param enable
609  * @param iCompletionCallbackWith
610  */
611 void setDeFogEnable(boolean enable, final CallbackWithNoParam
    iCompletionCallbackWith);
612 /**
613  * 获取雾透强度值
614  */
615 void setDeFogStrength(int strength, final CallbackWithNoParam
    iCompletionCallbackWith);
616
617 /**
618  * 获取雾透参数信息
619  */
620 void getDeFogParams(final CallbackWithOneParam<DeFogParam>
    iCompletionCallbackWith);
621
622 /**
623  * 设置视频显示模式
624  * @param displayType 主屏显示模式
625  * @param callback
626  */
627 void setDisplayVideo(DisplayType displayType, CallbackWithNoParam callback);
628
629 /**
630  * 设置视频显示模式
631  * @param mainType 主屏显示模式
632  * @param smallType 小屏显示模式
633  * @param location 显示位置
634  * @param callback
635  */
636 void setDisplayVideo(DisplayType mainType, DisplayType smallType,
    DisplayLocation location, CallbackWithNoParam callback);
637
638 /**
639  * 获取视频显示模式

```

```

640 * @param callback
641 */
642 void getDisplayVideo(CallbackWithOneParam<DisplayType> callback);
643
644 /**
645 * 设置小屏显示模式
646 * @param displayType 小屏显示模式
647 * @param location 显示位置
648 * @param callback
649 */
650 void setSmallDisplayVideo(DisplayType displayType, DisplayLocation location,
        CallbackWithNoParam callback);
651
652 /**
653 * 设置小视频显示开关
654 * @param state
655 * @param location
656 * @param callback
657 */
658 void setOpenSmallDisplayVideo(AutelSwitchState state, DisplayLocation
        location, CallbackWithNoParam callback);
659
660 /**
661 * 获取小屏显示信息
662 * @param callback
663 */
664 void getSmallDisplayVideo(CallbackWithOneParam<DisplayInfo> callback);
665
666 /**
667 * 设置双屏显示模式
668 *
669 * @param dualScreenMode 0 为5, 4并排显示, 1为双屏平铺4: 3 (单相机 2:3)
670 * @return
671 */
672 void setDualScreenMode(int dualScreenMode, CallbackWithNoParam callback);
673
674 /**
675 * 设置双屏显示模式
676 *
677 * @param zoomLink 0 关闭使能变焦, 1 使能联动变焦
678 * @return
679 */
680 void setZoomLinkMode(int zoomLink, CallbackWithNoParam callback);
681
682 /**
683 * 设置ICR开关
684 * @param state

```



```
685 * @param callback
686 */
687 void setICREnable(AutelSwitchState state, CallbackWithNoParam callback);
688
689 /**
690 * 获取ICR开关
691 * @param callback
692 */
693 void getICREnable(CallbackWithOneParam<AutelSwitchState> callback);
694
695 /**
696 * 设置点测光区域
697 * @param x 横坐标
698 * @param y 纵坐标
699 * @param callback
700 */
701 void setSpotMeteringArea(int x, int y, CallbackWithNoParam callback);
702
703 /**
704 * Returns the parameter range manager of XB015 camera.
705 *
706 * @return the parameter range manager of XB015 camera.
707 */
708 MultiParameterRangeManager getParameterRangeManager();
709
710 /**
711 * 设置电子防抖使能开关
712 * @param enable
713 * @param callback
714 */
715 void setDISEnable(boolean enable, CallbackWithNoParam callback);
716
717 /**
718 * 获取电子防抖使能开关
719 * @param callback
720 */
721 void getDISEnable(CallbackWithOneParam<Boolean> callback);
722
723 /**
724 * 设置显示模式
725 * @param displayMode
726 * @param callback
727 */
728 void setDisplayMode(DisplayMode displayMode, CallbackWithNoParam callback);
729
730 /**
731 * 获取显示模式
732 * @param callback
733 */
```

```

732 void getDisplayMode(CallbackWithOneParam<DisplayMode> callback);
733 /**
734  * 设置热成像相机色彩模式
735  *
736  * @param irColor
737  * @param callback
738  */
739 void setIrColor(IrColor irColor, CallbackWithNoParam callback);
740 /**
741  * 获取热成像相机色彩模式
742  *
743  * @param callback
744  */
745 void getIrColor(CallbackWithOneParam<IrColor> callback);
746 /**
747  * 设置热成像FFC 刷快门
748  *
749  * @param type
750  * @param callback
751  */
752 void setIrFlushShutter(int type, CallbackWithNoParam callback);
753
754 /**
755  * 待翻译 设置热成像相机图像模式
756  *
757  * @param imageMode
758  * @param contrast
759  * @param brightness
760  * @param callback
761  */
762 void setIrImageMode(ImageMode imageMode, int contrast, int brightness,
763 CallbackWithNoParam callback);
764 /**
765  * 获取热成像相机图像模式
766  *
767  * @param callback
768  */
769 void getIrImageMode(CallbackWithOneParam<IrImageModeParam> callback);
770
771 /**
772  * 待翻译 设置热成像相机图像增强
773  *
774  * @param enable 开关使能
775  * @param strength 强度
776  * @param callback
777  */

```

```
778 void setIrEnhance(boolean enable, int strength, CallbackWithNoParam callback);
779
780 /**
781  * 获取热成像相机图像增强
782  *
783  * @param callback
784  */
785 void getIrEnhance(CallbackWithOneParam<IrEnhanceParam> callback);
786
787 /**
788  * 待翻译 设置热成像相去噪开关
789  *
790  * @param enable 开关使能
791  * @param callback
792  */
793 void setIrDeNoise(boolean enable, CallbackWithNoParam callback);
794
795 /**
796  * 获取热热成像相机去噪开关
797  *
798  * @param callback
799  */
800 void getIrDeNoise(CallbackWithOneParam<Boolean> callback);
801
802 /**
803  * 待翻译 设置热成像相机增益
804  *
805  * @param mode 增益模式
806  * @param callback
807  */
808 void setIrGain(IrGainMode mode, CallbackWithNoParam callback);
809
810 /**
811  * 获取热热成像相机增益
812  *
813  * @param callback
814  */
815 void getIrGain(CallbackWithOneParam<IrGainMode> callback);
816
817 /**
818  * 待翻译 设置热成像等温线
819  *
820  * @param mode 等温线使能
821  * @param callback
822  */
823 void setIrIsoTherm(IrIsoThermMode mode, int hotThread, int coldThread,
    CallbackWithNoParam callback);
```

```
824
825 /**
826  * 获取热成像等温线信息
827  *
828  * @param callback
829  */
830 void getIrIsoTherm(CallbackWithOneParam<IrThresholdParam> callback);
831
832 /**
833  * 待翻译 设置热成像测温参数
834  *
835  * @param params
836  * @param callback
837  */
838 void setIrTemperatureParams(IrTempetureParams params, CallbackWithNoParam
callback);
839
840 /**
841  * 获取热成像测温参数信息
842  *
843  * @param callback
844  */
845 void getIrTemperatureParams(CallbackWithOneParam<IrTempetureParams> callback);
846
847 /**
848  * 设置热成像温度阈值参数
849  *
850  * @param params
851  * @param callback
852  */
853 void setIrTemperatureWarningParams(IrTemperatureWarnParams params,
CallbackWithNoParam callback);
854
855 /**
856  * 获取热成像温度阈值参数
857  *
858  * @param callback
859  */
860 void
getIrTemperatureWarningParams(CallbackWithOneParam<IrTemperatureWarnParams>
callback);
861
862 /**
863  * 设置热成像辐射率
864  *
865  * @param emit
866  * @param callback
```

```

867 */
868 void setIrTemperatureEmit(int emit, CallbackWithNoParam callback);
869
870 /**
871  * 获取热成像辐射率
872  *
873  * @param callback
874  */
875 void getIrTemperatureEmit(CallbackWithOneParam<Integer> callback);

```

1.3在相机拍照/录像前，我们可以使用AutelBaseCamera对相机进行一系列的参数设置，设置接口的回调函数分别为CallbackWithNoParam、CallbackWithOneParam：

```

1 public interface CallbackWithNoParam extends FailedCallback {
2     void onSuccess();
3 }
4
5 public interface CallbackWithOneParam<T> extends FailedCallback {
6     void onSuccess(T var1);
7 }
8
9 public interface FailedCallback {
10    /**
11     * Invokes this method when call back has failed.
12     * @param error the error object.
13     */
14    void onFailure(AutelError error);
15 }

```

1.4相机工作状态WorkState类介绍

```

1 WorkState {
2     //空闲状态
3     IDLE("idle", "Idle"),
4     //拍照
5     CAPTURE("capture", "TakingPhoto"),
6     //录像
7     RECORD("record", "Recording"),
8     //录像过程中拍照
9     RECORD_PHOTO_TAKING("record_taking", "Recording_taking"),
10    //未知状态
11    UNKNOWN("unknown", "unknown");
12 }

```

1.5相机模式设置，相机支持的模式

可以根据需求设置切换相机的模式，设置方法如下

```
1  protected MediaMode mediaMode = MediaMode.SINGLE;
2  AutelBaseCamera baseCamera = ((CameraActivity)
   getActivity()).getCurrentCamera();
3  baseCamera.setMediaMode(mediaMode, new CallbackWithNoParam() {
4      @Override
5      public void onFailure(AutelError error) {
6          logOut("setMediaMode description " +
   error.getDescription());
7      }
8
9      @Override
10     public void onSuccess() {
11         logOut("setMediaMode state onSuccess");
12     }
13 });
14
```

MediaMode参数说明:

```
1  //单拍
2  SINGLE("singal", "Single")
3  //录像
4  VIDEO("video", "Record")
5  //定时拍摄
6  TIMELAPSE("timelapse", "Timelapse")
7  //连拍
8  BURST("burst", "Burst")
9  //AEB
10 AEB("aeb", "AEB")
11 //HDR
12 HDR("hdr", "HDR")
13 //MFNR 纯净夜拍
14 MFNR("mfnr", "MFNR")
15 //延时拍摄
16 MOTION_DELAY_SHOT("MotionDelayShot", "MotionDelayShot")
17 //未知工作模式
18 UNKNOWN("unknown", "");
```

1.6实现拍摄功能

启动拍照之前，需要对下面几种状态进行判断：

```
1  /**
2      * 判断相机是否处于拍照模式下
3      */
4
5      MediaMode mode = MediaMode.find(getCameraMode());
6      if (MediaMode.BURST != mode &&
7          MediaMode.AEB != mode &&
8          MediaMode.SINGLE != mode &&
9          MediaMode.TIMELAPSE != mode) {
10         //非拍照模式
11         return;
12     }
13
14     /**
15      * 检查是否处于空闲状态
16      */
17     WorkState cameraWorkState = WorkState.find(getWorkStatus());
18     if (WorkState.IDLE != cameraWorkState) {
19         //相机正在工作
20         return;
21     }
22
23     /**
24      * 检查SDCard是否正常
25      */
26     SDCardState sdCardState = SDCardState.find(getSdCardStatus());
27     if (sdCardState != SDCardState.CARD_READY && sdCardState !=
SDCardState.LOW_SPEED_CARD) {
28         //SD卡异常
29         return;
30     }
```

实现拍照功能：

```
1  AutelBaseCamera baseCamera = ((CameraActivity)
    getActivity()).getCurrentCamera();
2  @Override
3      public void onClick(View v) {
4          baseCamera.startTakePhoto(new CallbackWithNoParam() {
5              @Override
```

```

6         public void onFailure(AutelError error) {
7             logOut("startTakePhoto " + error.getDescription());
8         }
9
10        @Override
11        public void onSuccess() {
12            logOut("startTakePhoto onSuccess");
13        }
14    });
15    }
16    });

```

1.7实现停止拍照

```

1  @Override
2      public void onClick(View v) {
3          baseCamera.startTakePhoto(new CallbackWithNoParam() {
4              @Override
5              public void onFailure(AutelError error) {
6                  logOut("startTakePhoto " + error.getDescription());
7              }
8
9              @Override
10             public void onSuccess() {
11                 logOut("startTakePhoto onSuccess");
12             }
13         });
14     }
15 }

```

1.8实现录像功能

在启动录像功能之前需要对以下几种状态进行判断：

```

1  /**
2      * 判断相机是否处于录像模式
3      */
4      MediaMode mode = MediaMode.find(getCameraMode());
5      if (MediaMode.VIDEO != mode) {
6          //非录像模式
7          return;
8      }
9  /**
10     * 检查是否处于空闲状态

```



```

11         */
12         WorkState cameraWorkState = WorkState.find(getWorkStatus());
13         if (WorkState.IDLE != cameraWorkState) {
14             //相机正在工作
15             return;
16         }
17
18         /**
19          * 检查SDCard是否正常
20          */
21         SDCardState sdCardState = SDCardState.find(getSdCardStatus());
22         if (sdCardState != SDCardState.CARD_READY && sdCardState !=
SDCardState.LOW_SPEED_CARD) {
23             //SD卡异常
24             return;
25         }
26

```

启动录像功能

```

1 baseCamera.startRecordVideo(new CallbackWithNoParam() {
2     @Override
3     public void onFailure(AutelError error) {
4         logOut("startRecordVideo " + error.getDescription());
5     }
6
7     @Override
8     public void onSuccess() {
9         logOut("startRecordVideo onSuccess");
10    }
11    });

```

1.9停止录像实现

```

1 baseCamera.startRecordVideo(new CallbackWithNoParam() {
2     @Override
3     public void onFailure(AutelError error) {
4         logOut("startRecordVideo " + error.getDescription());
5     }
6
7     @Override
8     public void onSuccess() {
9         logOut("startRecordVideo onSuccess");

```

```
10     }  
11     });
```

至此，SDK拍摄录像的使用已经介绍完成，希望您喜欢本教程，并继续关注我们的下一个教程！

2、相机参数设置实现教程

2.1概述

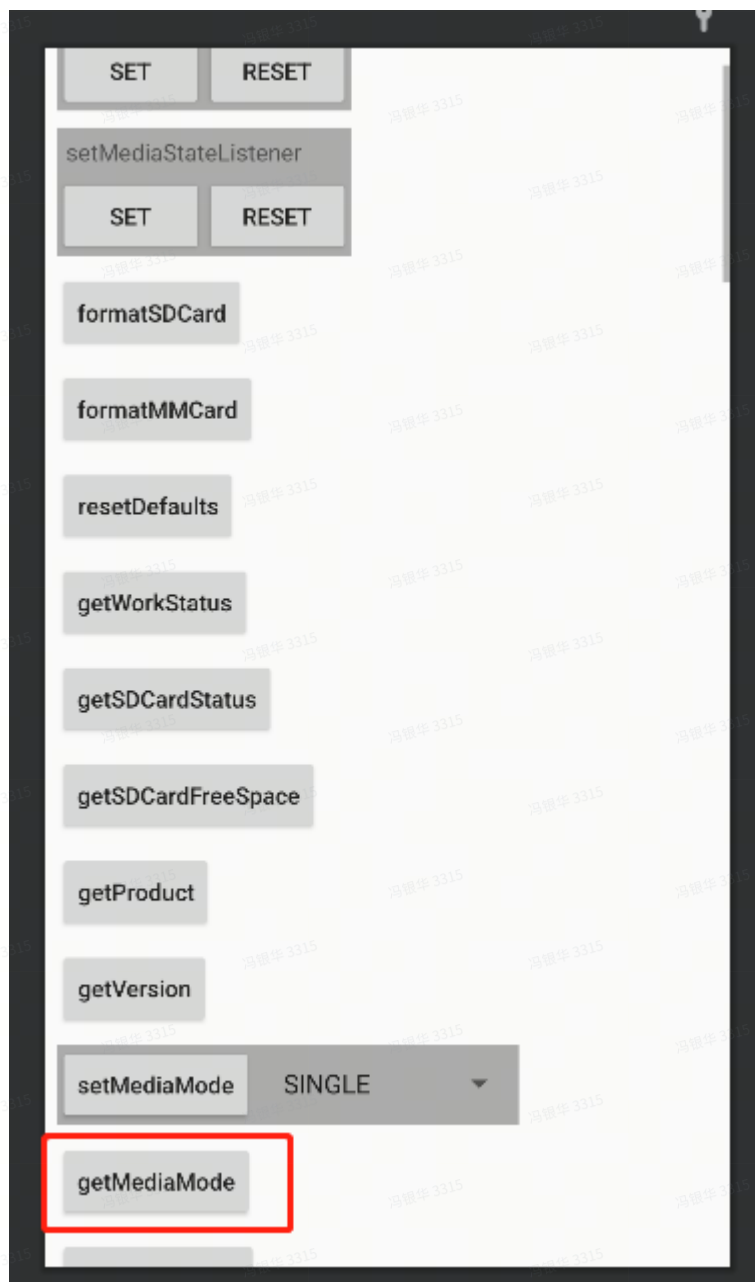
使用Mobile SDK 设定相机参数，可以选择很多不同的参数设置，本教程旨在帮助您基本了解MobileSDK 相机参数相关接口的使用。我们将分2个步骤：1，从SDK调起 AutelCameraManager(或者其他派生类) 2，相机参数设置。注：相机功能要基于SDK集成之后，首要参考SDK入门文档。

2.2功能实现

在CameraDFFragment中获取相机管理器

相机	管理器	相机类型
XT708	<i>MultiParameterRangeManager</i>	<i>AutelMultiCamera</i>
XT710	<i>MultiParameterRangeManager</i>	<i>AutelMultiCamera</i>
XT711	<i>MultiParameterRangeManager</i>	<i>AutelMultiCamera</i>
XT713	<i>MultiParameterRangeManager</i>	<i>AutelMultiCamera</i>
XT714	<i>MultiParameterRangeManager</i>	<i>AutelMultiCamera</i>
XT715	<i>MultiParameterRangeManager</i>	<i>AutelMultiCamera</i>
XT717	<i>MultiParameterRangeManager</i>	<i>AutelMultiCamera</i>

2.3函数调用



当我们操作了setMediaMode 代码会做什么呢？如下所示：

```
1 view.findViewById(R.id.getMediaMode).setOnClickListener(new
  View.OnClickListener() {
2     @Override
3     public void onClick(View v) {
4         autelMultiCamera.getMediaMode(new CallbackWithOneParam<MediaMode>() {
5             @Override
6             public void onSuccess(final MediaMode data) {
7                 logOut("getMediaMode " + data);
8                 Activity activity = getActivity();
9                 if (null != activity)
10                     if (null != currentVideoResolutionAndFps) {
11                         shutterSpeedAdapter.setData(rangeManager.getCameraShutterSpeed());
12                         shutterList.setAdapter(shutterSpeedAdapter);
```

```

13         }
14     }
15
16     @Override
17     public void onFailure(AutelError error) {
18         logOut("getMediaMode " + error.getDescription());
19     }
20 });
21 }
22 });

```

会调用autelMultiCamera，rangeManager身上的函数。下面我们介绍一下相机参数具体有哪些：

2.4 部分接口介绍

字段表2.4.1CameraProduct

```

1  /**
2   * The current camera type is XT701.
3   */
4  XT701("XT701") {
5  },
6  /**
7   * The current camera type is XT702.
8   */
9  XT702("XT702") {
10 },
11 /**
12  * The current camera type is XT703.
13  */
14  XT703("XT703") {
15  },
16  /**
17   * The current camera type is XT704.
18   */
19  XT704("XT704") {
20  },
21  /**
22   * The current camera type is XT706.
23   */
24  XT706("XT706") {
25  },
26  /**
27   * The current camera type is XT708.
28   */
29  XT708("XT708") {

```

```
30 },
31 /**
32  * The current camera type is XT709.
33  */
34 XT709("XT709") {
35 },
36 /**
37  * The current camera type is XT710.
38  */
39 XT710("XT710") {
40 },
41 /**
42  * The current camera type is XT711.
43  */
44 XT711("XT711") {
45 },
46 /**
47  * The current camera type is XT712.
48  */
49 XT712("XT712") {
50 },
51 /**
52  * The current camera type is XT713.
53  */
54 XT713("XT713") {
55 },
56 /**
57  * The current camera type is XT714.
58  */
59 XT714("XT714") {
60 },
61 /**
62  * The current camera type is XT715.
63  */
64 XT715("XT715") {
65 },
66 /**
67  * The current camera type is XT717.
68  */
69 XT717("XT717") {
70 },
71 /**
72  * The current camera type is XL718.
73  */
74 XL718("XL718") {
75 },
76 /**
```

```

77  * The current camera type is unknown.
78  */
79  UNKNOWN("UNKNOWN") {
80  };

```

字段表2.4.2接口BaseStateInfo

```

1  /**
2   * This interface is used to provide current state information of the camera.
3   */
4
5  public interface BaseStateInfo {
6      /**
7       * Get the camera type.
8       * @return the camera type.
9       */
10     CameraProduct getType();
11
12     /**
13      * Get the camera working state.
14      * @return the camera working state.
15      */
16     WorkState getWorkState();
17
18     /**
19      * Get the media mode of the camera.
20      * @return the media mode of the camera.
21      */
22     MediaMode getMediaMode();
23
24     /**
25      * Get the SD card state of the camera.
26      * @return the SD card state of the camera.
27      */
28     SDCardState getSDCardState();
29
30     /**
31      * Gets the color style.
32      * @return the color style.
33      */
34     ColorStyle getColorStyle();
35
36     /**
37      * Gets the white balance.
38      * @return the white balance.

```

```
39     */
40
41     WhiteBalance getWhiteBalance();
42
43     /**
44      * Gets the auto exposure state.
45      * @return the auto exposure state.
46      */
47
48     AutoExposureLockState getAutoExposureLockState();
49
50     /**
51      * Tells whether the histogram is enabled or not.
52      * @return enable the histogram or not.
53      */
54
55     boolean isHistogramEnable();
56
57     /**
58      * Gets the exposure mode.
59      * @return the exposure mode.
60      */
61
62     ExposureMode getExposureMode();
63
64     /**
65      * Gets the photo style value.
66      * @return the photo style value.
67      */
68
69     PhotoStyle getPhotoStyle();
70
71     /**
72      * Tells whether the subtitle is enabled or not.
73      * @return enable the subtitle or not.
74      */
75
76     boolean isSubtitleEnable();
77
78     /**
79      * Gets the camera anti flicker value.
80      * @return the camera anti flicker value.
81      */
82
83     AntiFlicker getAntiFlicker();
84
85     /**
```

```

86      * Gets the metering mode.
87      * @return the metering mode.
88      */
89      MeteringMode getMeteringMode();
90
91      /**
92      * Gets the video standard.
93      * @return the video standard.
94      */
95
96      VideoStandard getVideoStandard();
97
98      /**
99      * Gets the video format of the camera.
100     * @return the video format of the camera.
101     */
102     VideoFormat getVideoFormat();
103
104     /**
105     * Gets the photo aspect ratio.
106     * @return the photo aspect ratio.
107     */
108     PhotoAspectRatio getPhotoAspectRatio();
109
110     /**
111     * Gets the photo format.
112     * @return the photo format.
113     */
114     PhotoFormat getPhotoFormat();
115
116     /**
117     * Gets the digital zoom scale.
118     * @return the digital zoom scale.
119     */
120     int getDigitalZoomScale();
121
122     /**
123     * Gets the video resolution and frame rate.
124     * @return the video resolution and frame rate.
125     */
126     VideoResolutionAndFps getVideoResolutionAndFrameRate();
127
128     /**
129     * Gets the number of pictures continuously taken for each shooting in
    burst mode.
130     * @return the number of pictures continuously taken for each shooting in
    burst mode.

```



```

131     */
132
133     PhotoBurstCount getBurstCount();
134
135     /**
136      * Gets the number of pictures continuously taken for each shooting in AEB
137      * mode.
138      * @return the number of pictures continuously taken for each shooting in
139      * AEB mode.
140      */
141
142     PhotoAEBCount getAEBCount();
143
144     /**
145      * Gets the time interval between taking two photos.
146      * @return the time interval between taking two photos.
147      */
148
149     PhotoTimelapseInterval getTimelapseInterval();
150
151     /**
152      * Gets the PIV mode of the camera.
153      * @return the PIV mode of the camera.
154      */
155
156     PIVMode getPIVMode();
157
158     /**
159      * Gets the timelapse interval of the auto PIV mode.
160      * @return the timelapse interval of the auto PIV mode.
161      */
162
163     VideoSnapshotTimelapseInterval getAutoPIVTimelapseInterval();
164
165     /**
166      * Gets the video encode format.
167      * @return the video encode format.
168      */
169
170     VideoEncodeFormat getVideoEncodeFormat();
171
172     /**
173      * 待翻译
174      * 获取图片质量
175      * @return 图片质量
176      */
177
178     PhotoQualityLevel getPhotoQuality();

```

```
176
177 /**
178  * 待翻译
179  * 获取SD卡剩余容量，MB为单位
180  * @return SD卡剩余容量
181  */
182 long getSDCardFreeSpace();
183
184 /**
185  * 存储位置
186  * @return 存储位置
187  */
188 SaveLocation getSaveLocation();
189
190 /**
191  * 物距
192  * @return 物距
193  */
194 int getFocusDistance();
195
196 /**
197  * 对焦模式
198  * @return 对焦模式
199  */
200 LensFocusMode getLensFocusMode();
201
202 /**
203  * 获取ISO模式
204  * @return ISO模式
205  */
206 ISOMode getISOMode();
207
208 /**
209  * 获取机载闪存状态
210  * @return 机载闪存状态
211  */
212 FlashCardStatus getFlashCardStatus();
213
214 /**
215  * 获取云台锁状态
216  * @return 云台锁状态
217  */
218 GimbalLockState getGimbalLockState();
219
220 /**
221  * 获取显示模式
222  * @return 显示模式
```

```
223     */
224     DisplayMode getDisplayMode();
225
226     /**
227     * 获取红外色彩
228     * @return 红外色彩
229     */
230     IrColor getIrColor();
231
232     /**
233     * 获取版本号
234     * @return 版本号
235     */
236     String getVersion();
237
238     /**
239     * 获取红外增益模式
240     * @return 红外增益模式
241     */
242     IrGainMode getIrGainMode();
243
244     /**
245     * 获取红外图像模式
246     * @return 红外图像模式
247     */
248     ImageMode getImageMode();
249
250     /**
251     * 获取红外图像模式参数
252     * @return 红外图像模式参数
253     */
254     IrImageModeParam getIrImageModeParam();
255
256     /**
257     * 获取红外温度模式
258     * @return 红外温度模式
259     */
260     IrTemperatureMode getIrTemperatureMode();
261
262     /**
263     * 获取红外等温模式
264     * @return 红外等温模式
265     */
266     IrIsoThermMode getIrIsoThermMode();
267
268     /**
269     * 获取红外增强参数
270     * @return 红外增强参数
271     */
272     IrEnhanceParam getImageEnhanceParams();
```

```
270
271 /**
272  * 获取红外位置
273  * @return 红外位置
274  */
275 IrPosition getIrPosition();
276 /**
277  * 获取DNR设置
278  * @return DNR设置
279  */
280 DnrSetting getDnrSetting();
281
282 /**
283  * 获取等温线参数信息
284  * @return 等温线参数信息
285  */
286 IrThresholdParam getIrThresholdParam();
287
288 /**
289  * 获取红外温度参数
290  * @return 红外温度参数
291  */
292 IrTempetureParams getIrTempetureParams();
293
294 /**
295  * 获取红外温度发射率
296  * @return 红外温度发射率
297  */
298 int getIrTemperatureEmit();
299
300 /**
301  * 获取去雾参数
302  * @return 去雾参数
303  */
304 DeFogParam getDeFogParam();
305
306 /**
307  * 获取ICR开关状态
308  * @return ICR开关状态
309  */
310 AutelSwitchState getIcrEnable();
311
312 /**
313  * 获取Dis信息
314  * @return Dis信息
315  */
316 DisInfo getDisInfo();
```

```

317
318     /**
319     * 获取红外温度告警参数
320     * @return 红外温度告警参数
321     */
322     IrTemperatureWarnParams getIrTemperatureWarnParams();
323
324     /**
325     * 获取红外辅助对接开关状态
326     * @return 红外辅助对接开关状态
327     */
328     boolean getMFFocusAssistEnable();
329
330     /**
331     * 获取红外NR开关状态
332     * @return 红外NR开关状态
333     */
334     boolean getIrNrEnable();
335 }

```

3、飞机参数设置功能实现

3.1概述

使用Mobile SDK 设定飞行参数，可以选择很多不同的参数设置，本教程旨在帮助您基本了解MobileSDK 飞机相关接口的使用。

我们将分2个步骤：

3.1.1从SDK调起 AutelFlyController (或者其他派生类)

3.1.2飞机参数设置。注：飞行功能要基于SDK集成之后，首要参考SDK入门文档。

3.2功能实现

3.2.1 从SDK集成过程中Autel获取product 对象

此处SDK集成时候做的，我们只是告知开发者BaseProduct对象的来处 (Autel.setProductConnectListener中)

```

1 Autel.setProductConnectListener(new ProductConnectListener() {
2     @Override
3     public void productConnected(BaseProduct product) {
4         // BaseProduct product

```

```

5      getApplicationContext().setCurrentProduct(product);
6      }
7      @Override
8      public void productDisconnected() {
9
10     }
11     });

```

3.2.2 Activity中使用product对象

以CruiserFlyController文件为例，其他的雷同

```

1  private CruiserFlyController mController;
2  @Override
3  protected AutelFlyController initController(BaseProduct product) {
4      mController = ((CruiserAircraft) product).getFlyController();
5      return mController;
6  }

```

3.2.3 具体调用mController对象方法

以激活设备功能为代表，其他方法雷同。

```

1  /**
2   * 激活设备
3   *
4   * @param view
5   */
6  public void activation(View view) {
7      mController.activation(new CallbackWithTwoParams<Boolean, String>() {
8          @Override
9          public void onSuccess(Boolean data1, String data2) {
10         logOut("激活 : " + data1 + " message : " + data2);
11     }
12
13     @Override
14     public void onFailure(AutelError error) {
15         logOut("激活失败 : " + error.getDescription());
16     }
17     });
18 }

```

3.3 参数设置及接口数据监听

参数设置会涉及到很多接口和参数，以下全部会列举出来。已经出现的接口和字段表，如果重复就不再列举。

3.3.1AutelFlyController.java 接口基类

说明：该接口用于提供飞行控制功能

```
1  /**
2   * This interface is used to provide flight control functions.
3   */
4
5  public interface AutelFlyController {
6      /**
7       * Sets the listener for the flight control warning state to monitor the
8       * real-time data.
9       *
10      * @param listener the listener will be cancelled when the callback is
11      * null.
12      */
13      void setWarningListener(CallbackWithTwoParams<ARMWarning,
14      MagnetometerState> listener);
15
16      /**
17       * Sets the listener for calibrating the compass to monitor the real-time
18       * data.
19       *
20       * @param listener the listener will be cancelled when the callback is
21       * null.
22       */
23      void setCalibrateCompassListener(final
24      CallbackWithOneParam<CalibrateCompassStatus> listener);
25
26      /**
27       * Sets beginner mode to be enabled or disabled.
28       *
29       * @param enable whether to enable beginner mode.
30       * @param callback the callback when set beginner mode to be enabled or
31       * disabled.
32       */
33      void setBeginnerModeEnable(boolean enable, CallbackWithNoParam callback);
34
35      /**
36       * Checks if beginner mode is enabled.
37       *
38       * @param callback the callback of turning on/off beginner mode.
39       */
40      void isBeginnerModeEnable(CallbackWithOneParam<Boolean> callback);
```

```
34
35  /**
36   * Sets the altitude limit for the aircraft.
37   *
38   * @param value the altitude limit, XStar's maximum flight altitude
   ranges from 30 meters to 800 meters.
39   * @param callback the callback when set the altitude limit for the
   aircraft.
40   */
41   void setMaxHeight(double value, CallbackWithNoParam callback);
42
43  /**
44   * Returns the altitude limit of the aircraft.
45   *
46   * @param callback the callback when get the altitude limit of the
   aircraft.
47   */
48   void getMaxHeight(CallbackWithOneParam<Float> callback);
49
50  /**
51   * Sets the distance limit for the aircraft.
52   *
53   * @param value the distance limit, XStar's maximum flight distance
   ranges from 30 meters to 500 meters.
54   * @param callback the callback when set the distance limit for the
   aircraft.
55   */
56   void setMaxRange(double value, CallbackWithNoParam callback);
57
58  /**
59   * Returns the distance limit of the aircraft.
60   *
61   * @param callback the callback when get the distance limit of the
   aircraft.
62   */
63   void getMaxRange(CallbackWithOneParam<Float> callback);
64
65  /**
66   * Sets the go-home altitude for the aircraft.
67   *
68   * @param value the go home altitude, XStar's go home altitude ranges
   from 30 meters to 200 meters.
69   * @param callback the callback when set the go-home altitude for the
   aircraft.
70   */
71   void setReturnHeight(double value, CallbackWithNoParam callback);
72
```



```

73  /**
74   * Returns the go-home altitude of the aircraft.
75   *
76   * @param callback the callback when get the go-home altitude of the
aircraft.
77  */
78  void getReturnHeight(CallbackWithOneParam<Float> callback);
79
80  /**
81   * Sets the horizontal speed limit for the aircraft.
82   *
83   * @param value the horizontal speed limit, XStar's maximum horizontal
speed ranges from 0 m/s to 15 m/s.
84   * @param callback the callback when set the horizontal speed limit for
the aircraft.
85   */
86  void setMaxHorizontalSpeed(double value, CallbackWithNoParam callback);
87
88  /**
89   * Returns the horizontal speed limit of the aircraft.
90   *
91   * @param callback the callback when get the horizontal speed limit of the
aircraft.
92   */
93  void getMaxHorizontalSpeed(CallbackWithOneParam<Float> callback);
94
95  /**
96   * Sets attitude mode to be enabled or disabled.
97   *
98   * @param enable whether to enable attitude mode.
99   * @param callback the callback when set attitude mode to be enabled or
disabled.
100  */
101  void setAttitudeModeEnable(boolean enable, CallbackWithNoParam callback);
102
103  /**
104   * Checks if attitude mode is enabled.
105   *
106   * @param callback the callback of turning on/off attitude mode.
107   */
108  void isAttitudeModeEnable(CallbackWithOneParam<Boolean> callback);
109
110  /**
111   * Sets the LED indicator state located on the aircraft.
112   *
113   * @param lamp the enum of the LED indicator state.

```

```
114      * @param callback the callback when set the LED indicator state located
115      * /
116      void setLedPilotLamp(NavigationControllLight lamp, CallbackWithNoParam
117      callback);
118      /**
119      * Returns the LED indicator state located on the aircraft.
120      *
121      * @param callback the callback when get the LED indicator state located
122      * /
123      void getLedPilotLamp(CallbackWithOneParam<NavigationControllLight>
124      callback);
125      /**
126      * Starts the compass calibration.
127      *
128      * @param callback the callback when start the compass calibration.
129      * /
130      void startCalibrateCompass(CallbackWithOneParam<CalibrateCompassStatus>
131      callback);
132      /**
133      * Takes off the aircraft with one press on the remote control.
134      * Note: it is only available when the aircraft is landed and is in GPS
135      * mode.
136      * @param callback the callback when take off the aircraft with one press
137      * /
138      void takeOff(CallbackWithOneParam<Pair<Boolean, FlightErrorState>>
139      callback);
140      /**
141      * Makes the aircraft go home.
142      *
143      * Note:
144      * This is available when the aircraft is not landing or landed.
145      * Also, there must be GPS signal, valid home point, normal compass and
146      * running motors.
147      *
148      * @param callback the callback when make the aircraft go home.
149      * /
150      void goHome(CallbackWithNoParam callback);
151      /**
```

```

152      * Makes the aircraft land descent.
153      * Note:
154      * This is available when the aircraft has taken off.
155      *
156      * @param callback the callback when make the aircraft land descent.
157      */
158      void land(CallbackWithNoParam callback);
159
160      /**
161      * Makes the aircraft land descent.
162      * Note:
163      * This is available when the aircraft has taken off.
164      * @param landType the landing type.
165      * @param params the landing parameters.
166      * @param callback the callback when make the aircraft land descent.
167      */
168      void land(LandType landType, LandingParams params, CallbackWithNoParam
callback);
169
170      /**
171      * 降落到预设备降点
172      * @param landIndex 降落点序号 从1开始~备降点的个数
173      * @param callback 回调
174      */
175      void land(int landIndex, CallbackWithNoParam callback);
176
177
178      /**
179      * Cancels the land operation.
180      * Note:
181      * This is available when the aircraft is landing.
182      *
183      * @param callback the callback when cancel the land operation.
184      */
185      void cancelLand(CallbackWithNoParam callback);
186
187      /**
188      * Cancels the go home operation and make the aircraft hover.
189      * Note:
190      * This is only available when the aircraft is in the go home process.
191      *
192      * @param callback the callback when cancel the go home operation and make
the aircraft hover.
193      */
194      void cancelReturn(CallbackWithNoParam callback);
195
196      /**

```

```

197      * Sets the current aircraft location as the home point.
198      * Note:
199      * This is available if all the following requirements are met:
200      * The motors are running, the aircraft is not in any intelligent mode,
the aircraft is not returning to the home point, GPS is valid.
201      *
202      * @param callback the callback when set the current aircraft location as
the home point.
203      */
204      void setAircraftLocationAsHomePoint(CallbackWithNoParam callback);
205
206      /**
207      * Sets the current phone location as the home point.
208      *
209      * @param lat the latitude value to be set.
210      * @param lng the longitude value to be set.
211      * Note:
212      * This is available if all the following requirements are
met:
213      * The motors are running, the aircraft is not in any
intelligent mode, the aircraft is not returning to the home point.
214      * GPS for the aircraft and phone is valid, the distance
between the phone and aircraft is within 5000 meters.
215      * @param callback the callback when set the current phone location as the
home point.
216      */
217      void setLocationAsHomePoint(double lat, double lng, CallbackWithNoParam
callback);
218
219      /**
220      * Returns the serial number of the aircraft.
221      *
222      * @param callback the callback when get the serial number of the aircraft.
223      */
224      void getSerialNumber(CallbackWithOneParam<String> callback);
225
226      /**
227      * Returns the version information of the aircraft.
228      *
229      * @param callback the callback when get the version information of the
aircraft.
230      */
231      void getVersionInfo(CallbackWithOneParam<FlyControllerVersionInfo>
callback);
232
233      /**
234      * Returns the version information of the FMU.

```

```

235     * @param callback the callback when get the version information of the
FMU.
236     */
237     void getFmuVersionInfo(CallbackWithOneParam<FmuControllerVersionInfo>
callback);
238
239
240     /**
241     * Returns the parameter range manager of the flight control.
242     *
243     * @return the parameter range manager of the flight control.
244     */
245     FlyControllerParameterRangeManager getParameterRangeManager();
246
247     /**
248     * Converts to ReactiveX interface.
249     *
250     * @return ReactiveX interface.
251     */
252     RxAutelFlyController toRx();
253
254     /**
255     * 设置起飞高度
256     * @param height 起飞高度
257     * @param callback 回调
258     */
259     void setTakeOffAltitude(int height, CallbackWithOneParam<Integer>
callback);
260
261     /**
262     * 获取起飞高度
263     * @param callback 回调
264     */
265     void getTakeOffAltitude(CallbackWithOneParam<Integer> callback);
266
267     /**
268     * 设置ADS-B ICAO
269     * @param data ICAO
270     * @param callback 回调
271     */
272     void setADS_B_ICAO(int data, CallbackWithNoParam callback);
273
274     /**
275     * 获取ADS-B ICAO
276     * @param callback 回调
277     */
277     void getADS_B_ICAO(CallbackWithOneParam<ADSBCmdInfo> callback);
278     /**

```

```

279     * 设置ADS-B 使能
280     * @param isEnabled 是否开启使能开关
281     * @param callback 回调
282     */
283     void enableADSBTX(boolean isEnabled, CallbackWithNoParam callback);
284
285     /**
286     * 设置飞控参数
287     * @param value 参数值
288     * @param cmd 参数名
289     * @param callback 回调
290     */
291     void updateFmuParamByInt(int value, String cmd,
292     CallbackWithOneParam<Integer> callback);
293
294     /**
295     * 获取飞控参数
296     * @param value 参数值
297     * @param cmd 参数名
298     * @param callback 回调
299     */
300     void updateFmuParamByFloat(float value, String cmd,
301     CallbackWithOneParam<Float> callback);
302
303     /**
304     * 获取飞控参数
305     * @param cmd 参数名
306     * @param callback 回调
307     */
308     void getFmuParamByInt(String cmd, CallbackWithOneParam<Integer> callback);
309
310     /**
311     * 获取gnss状态
312     * @param callback 回调
313     */
314     void getNoGnssState(CallbackWithOneParam<Integer> callback);
315
316     /**
317     * 机翼执行机构参数查询
318     *
319     * @param type 机翼执行机构类型
320     * @param callback 回调
321     */
322     void getActuatorParam(ActuatorType type,
323     CallbackWithOneParam<ActuatorParam> callback);
324
325     /**
326     * 获取飞机维护时长
327     * @param callback 回调

```

```

323     */
324     void getMaintainDur(CallbackWithOneParam<Integer> callback);
325
326     /**
327     * 校准飞机位置
328     * @param coordinate3D 位置
329     * @param callback 回调
330     */
331     void calibrateUavLocation(AutelCoordinate3D
coordinate3D, CallbackWithNoParam callback);

```

3.3.2 CruiserFlyController 飞控接口类

```

1  /**
2  * This interface is used to provide flight control functions for EVO.
3  */
4  public interface CruiserFlyController extends AutelFlyController,
CruiserVisual {
5
6      /**
7      * 飞机跟随
8      *
9      * @param data      0-不跟随, 1-跟随
10     * @param callback 数据回调
11     */
12     void setAircraftFollowing(int data, CallbackWithOneParam<Float> callback);
13
14     /**
15     * 获取飞机跟随状态
16     *
17     * @param iCompletionCallbackWith 0-不跟随, 1-跟随
18     */
19     void getAircraftFollowing(CallbackWithOneParam<Float>
iCompletionCallbackWith);
20
21     /**
22     * 开启地形辅助避障功能
23     *
24     * @param data      0-关闭, 1-打开
25     * @param callback 数据回调
26     */
27     void setObstacleAvoidanceSwitch(int data, CallbackWithOneParam<Float>
callback);
28
29     /**

```

```

30     * 获取地形辅助避障开关状态
31     *
32     * @param callback 数据回调
33     */
34     void getObstacleAvoidanceSwitch(CallbackWithOneParam<Float> callback);
35
36     /**
37     * 设置飞机跟踪坐标上报监听
38     * @param listener 回调
39     */
40     void setTrackCoordinateListener(final
CallbackWithOneParam<TrackCoordinate> listener);
41
42     /**
43     * 设置指南针校准进度监听
44     * @param listener 回调
45     */
46     void
setCalibrateCompassProgressListener(CallbackWithOneParam<CompassProgress>
listener);
47
48     /**
49     * 设置空速计校准进度监听
50     * @param listener 回调
51     */
52     void setCalibrateAirSpeedListener(final
CallbackWithOneParam<AirSpeedProgress> listener);
53
54     /**
55     * 设置起飞结果状态监听
56     * @param listener 回调
57     */
58     void setTakeOffStateListener(final CallbackWithOneParam<FlightErrorState>
listener);
59
60     /**
61     * Sets the listener for the flight control to monitor the real-time data.
62     *
63     * @param listener the listener will be cancelled when the callback is
null.
64     */
65     void setFlyControllerInfoListener(final
CallbackWithOneParam<CruiserFlyControllerInfo> listener);
66
67     /**
68     * Sets the listener for the flight control to monitor parameter variation
69     *

```



```

70      * @param listener the listener will be cancelled when the callback is null
71      */
72      void setFlyControllerParameterChangedListener(final
CallbackWithTwoParams<FmuParameterEvent, Float> listener);
73      /**
74      * 设置FMU运行信息监听
75      * @param listener 回调
76      */
77      void setFmuRunningInfoListener(final CallbackWithOneParam<FMURunningInfo>
listener);
78
79      /**
80      * 设置在线规划信息监听
81      * @param listener 回调
82      */
83      void setOnlinePlanningListener(final
CallbackWithOneParam<OnlinePlanningInfo> listener);
84
85      /**
86      * 设置ADS-B飞机信息监听
87      * @param callbackWith 回调
88      */
89      void setADSBBVehicleInfoListener(CallbackWithOneParam<ADSBBVehicleDataInfo>
callbackWith);
90
91      /**
92      * Changes the aircraft to the arm mode.
93      *
94      * @param callback the callback when change the aircraft to the arm mode.
95      */
96      void droneArmed(CallbackWithNoParam callback);
97
98      /**
99      * Changes the aircraft to the disarm mode.
100      *
101      * @param callback the callback when change the aircraft to the disarm
mode.
102      */
103      void droneDisarmed(CallbackWithNoParam callback);
104
105      /**
106      * Returns the parameter range manager of the flight control.
107      *
108      * @return the parameter range manager of the flight control.
109      */
110      EvoFlyControllerParameterRangeManager getParameterRangeManager();
111

```

```

112  /**
113   * 待翻译
114   * 设置机头转动方向
115   * direct -100 ~ 100 负值标示机头朝左，机头朝右
116   * Sets aircraft heading di
117   *
118   * @param direct Range from -100 to 100, 0 indicates front, negative
number indicates counterclockwise rotation speed; positive number indicates
clockwise rotation speed
119   * @param callback the callback when set heading direction for the
aircraft.
120   */
121  void setAircraftHeadingDirection(int direct, CallbackWithNoParam callback);
122
123  /**
124   * 获取指南针校准结果
125   * @param callback 回调
126   */
127  void getCalibrateCompassResult(CallbackWithOneParam<CalibCompass>
callback);
128
129  /**
130   * 开始空速计校准
131   *
132   * @param callback 回调
133   */
134  void startAirSpeedCalibrate(CallbackWithNoParam callback);
135
136  /**
137   * 获取空速计校准结果
138   * @param callback 回调
139   */
140  void getCalibrateAirSpeedResult(CallbackWithOneParam<AirSpeed> callback);
141
142  /**
143   * 设置最低飞行高度
144   * @param value 高度
145   * @param callback 回调
146   */
147  void setMinAltitude(double value, CallbackWithNoParam callback);
148
149  /**
150   * 设置激活状态
151   * @param value 0-未激活 1-激活
152   * @param callback 回调
153   */
154  void setActivateState(int value, CallbackWithNoParam callback);

```

```

155
156 /**
157  * 设置失联动作
158  * @param data 失联动作
159  * @param callback 回调
160  */
161 void setLostContactAction(int data, CallbackWithOneParam<Integer>
callback);
162
163 /**
164  * 获取失联动作
165  * @param callback 回调
166  */
167 void getLostContactAction(CallbackWithOneParam<Integer> callback);
168
169 /**
170  * 切换飞行模式
171  * @param callback 回调
172  */
173 void changeFlyMode(final CallbackWithNoParam callback);
174
175 /**
176  * 获取最小高度
177  * @param callback 回调
178  */
179 void getMinAltitude(CallbackWithOneParam<Float> callback);
180
181 /**
182  * 设置安全高度
183  * @param height 高度
184  * @param callback 回调
185  */
186 void setSafeHeight(int height, CallbackWithOneParam<Float> callback);
187
188 /**
189  * 获取安全高度
190  * @param callback 回调
191  */
192 void getSafeHeight(CallbackWithOneParam<Float> callback);
193
194 /**
195  * 开始自检
196  * @param value 检测模块
197  * @param callback 回调
198  */
199 void autoSafeCheck(ModelType value, CallbackWithNoParam callback);
200
201 /**
202  * 获取自检结果状态
203  * @param value 检测模块

```

```

201     * @param callback 回调
202     */
203     void getAutoSafeCheck(ModelType value, CallbackWithOneParam<AutoSafeState>
callback);
204
205     /**
206     * 设置应急降落点
207     * @param lat 纬度
208     * @param lng 经度
209     * @param callback 回调
210     */
211     void setLocationAsUrgency(double lat, double lng, CallbackWithNoParam
callback);
212
213     /**
214     * 设置飞机飞行快速任务点
215     * @param lat 纬度
216     * @param lng 经度
217     * @param alt 高度
218     * @param radius 半径
219     * @param speed 速度
220     * @param direction 方向
221     * @param distance 距离
222     * @param callback 回调
223     */
224     void startQuickMission(double lat, double lng, int alt, int radius, float
speed, int direction, int distance, CallbackWithNoParam callback);
225
226     /**
227     * 设置飞机航线任务飞行高度
228     * @param alt 高度
229     * @param callback 回调
230     */
231     void setWayLineAltitude(int alt, CallbackWithNoParam callback);
232
233     /**
234     * 停止快速任务 飞向指定航点
235     * @param waypointID 航点ID
236     * @param callback 回调
237     */
238     void stopQuickMission(int waypointID, CallbackWithNoParam callback);
239
240     /**
241     * 设置兴趣点坐标位置
242     * @param lat 纬度
243     * @param lng 经度
244     * @param alt 高度

```

```
245     * @param callback 回调
246     */
247     void setLocationAsInterest(double lat, double lng, int alt,
    CallbackWithNoParam callback);
248
249     /**
250     * 检测单个高程数据在飞机上是否存在
251     * @param path 文件路径
252     * @param callback 回调
253     */
254     void checkElevation(String path, CallbackWithOneParam<Boolean> callback);
255
256     /**
257     * 检测多个高程数据在飞机上是否存在
258     * @param path 文件路径
259     * @param callback 回调
260     */
261     void checkElevationExist(List<String> path,
    CallbackWithOneParam<List<String>> callback);
262
263     /**
264     * 上传高程数据到飞机
265     * @param elationPath 文件路径
266     * @param callback 上传文件的进度回调
267     */
268     void uploadElevationData(String elationPath,
    CallbackWithOneParamProgress<Float> callback);
269
270     /**
271     * 检测文件是否存在飞机上
272     * @param fileType 文件类型
273     * @param file 文件
274     * @param callback 回调
275     */
276     void checkFileData(FileDataType fileType, File file,
    CallbackWithOneParam<Boolean> callback);
277
278     /**
279     * 停止检测文件数据是否在飞机端
280     */
281     void stopCheckFileData();
282
283     /**
284     * 上传文件数据到飞机
285     * @param fileType 文件类型
286     * @param file 文件
287     * @param callback 上传文件的进度回调
```

```

288     */
289     void uploadFileData(FileDataType fileDataType, File file,
    CallbackWithOneParamProgress<Float> callback);
290
291     /**
292     * 上传文件数据到飞机端 通过TCP上传
293     * @param filePath 文件路径
294     * @param callback 回调
295     */
296     void uploadFileDataByTcp(String filePath,
    CallbackWithOneParamProgress<Float> callback);
297
298     /**
299     * 通知文件任务上传完成状态给飞机
300     * @param fileDataType 文件类型
301     * @param missionID 任务ID
302     * @param callbackWithOneParam 回调
303     */
304     void notifyFileStatus(FileDataType fileDataType, int missionID,
    CallbackWithOneParam<Boolean> callbackWithOneParam);
305
306     /**
307     * 设置移动基站位置
308     *
309     * @param latitude 纬度
310     * @param longitude 经度
311     * @param altitude 高度
312     * @param callback 回调
313     */
314     void setBaseStationLocation(double latitude, double longitude, double
    altitude, CallbackWithOneParam<Boolean> callback);
315
316     /**
317     * 设置基站类型
318     *
319     * @param type 基站类型
320     * @param callback 回调
321     */
322     void setBaseStationType(StationType type, CallbackWithOneParam<Boolean>
    callback);
323
324     /**
325     * 获取基站类型
326     *
327     * @param callback 回调
328     */
329     void getBaseStationType(CallbackWithOneParam<StationType> callback);

```

```

330
331     /**
332      * Converts to ReactiveX interface.
333      *
334      * @return ReactiveX interface.
335      */
336     RxCruiserFlyController toRx();
337
338
339     /**
340      * 设置当前国家固定禁飞区是否禁飞
341      *
342      * @param value true-禁飞 false-不禁飞
343      * @param callback 返回设置结果
344      */
345     void setNFZState(boolean value, final CallbackWithOneParam<Integer>
callback);
346
347     /**
348      * 获取国家禁飞区是否禁飞
349      *
350      * @param callback true-禁飞 false-不禁飞
351      */
352     void getNFZState(CallbackWithOneParam<Boolean> callback);
353
354     /**
355      * 待 检查禁飞区数据是否需要更新
356      *
357      * @param country 临时禁飞区时为文件名, 固定禁飞区时为国家命名的名字
358      * @param md5 文件在md5值
359      * @param callback 返回是否需要更新禁飞区数据
360      */
361     void checkNFZ(String country, String md5, CallbackWithOneParam<Boolean>
callback);
362
363     /**
364      * 待 上传文件数据给飞控端
365      *
366      * @param filePath 文件路径
367      * @param type 文件类型
368      * @param callback 返回上传文件的进度回调
369      */
370     void uploadFileData(String filePath, FileDataType type,
CallbackWithOneParamProgress<Float> callback);
371
372
373     /**
374      * 获取 跟踪类型

```

```

374      *
375      * @param callback 回调
376      */
377      void getTrackType(CallbackWithOneParam<TrackType> callback);
378
379      /**
380      * 设置跟踪类型
381      *
382      * @param trackType 跟踪类型
383      * @param callback 回调
384      */
385      void setTrackType(TrackType trackType, CallbackWithNoParam callback);
386
387      /**
388      * 设置临时观察点
389      *
390      * @param enable 是否开启
391      * @param lat 纬度
392      * @param lng 经度
393      * @param altitude 高度
394      * @param radius 半径
395      * @param callback 回调
396      */
397      void setTempSurvey(boolean enable, double lat, double lng, int altitude,
398      int radius, CallbackWithNoParam callback);
399
400      /**
401      * 切换航点任务飞行
402      * @param switchWayPointType 1:开始任务 2:开始临时任务 3:返回任务 4:返回临时
403      任务
404      * @param missionID 航点任务的missionID
405      * @param positionId 航点序号ID
406      * @param displayId 航点序号ID
407      * @param callback 回调
408      */
409      void switchWaypoint(SwitchWayPointType switchWayPointType, int missionID,
410      int positionId, int displayId, CallbackWithNoParam callback);
411
412      /**
413      * 设置跟踪半径参数
414      *
415      * @param radius 跟踪半径
416      * @param callback 回调
417      */
418      void setTrackRadius(float radius, CallbackWithOneParam<Float> callback);
419
420      /**

```



```

418     * 设置跟踪距离参数
419     *
420     * @param distance 跟踪距离
421     * @param callback 回调
422     */
423     void setTrackDistance(float distance, CallbackWithOneParam<Float>
callback);
424
425     /**
426     * 设置跟踪角度类型参数
427     *
428     * @param angleType 角度类型 1: 绝对角度 2: 相对角度
429     * @param callback 回调
430     */
431     void setTrackAngleType(float angleType, CallbackWithOneParam<Float>
callback);
432
433     /**
434     * 设置跟踪偏移角度参数
435     *
436     * @param angleValue 偏移角度(0~360度)
437     * @param callback 回调
438     */
439     void setTrackAngleValue(float angleValue, CallbackWithOneParam<Float>
callback);
440
441     /**
442     * 设置跟踪跟踪环绕方向参数
443     *
444     * @param dir 跟踪环绕方向 1: 逆时针 0: 自动 -1: 顺时针
445     * @param callback 回调
446     */
447     void setTrackSurroundOrientation(int dir, CallbackWithOneParam<Integer>
callback);
448
449     /**
450     * 获取跟踪半径参数
451     *
452     * @param callback 回调
453     */
454     void getTrackRadius(CallbackWithOneParam<Float> callback);
455
456     /**
457     * 获取跟踪距离参数
458     *
459     * @param callback 回调
460     */

```

```

461 void getTrackDistance(CallbackWithOneParam<Float> callback);
462
463 /**
464  * 获取跟踪角度类型参数 角度类型 1: 绝对角度 2: 相对角度
465  *
466  * @param callback 回调
467  */
468 void getTrackAngleType(CallbackWithOneParam<Float> callback);
469
470 /**
471  * 获取跟踪偏移角度参数
472  *
473  * @param callback 回调
474  */
475 void getTrackAngleValue(CallbackWithOneParam<Float> callback);
476
477 /**
478  * 获取跟踪跟踪环绕方向参数
479  *
480  * @param callback 回调
481  */
482 void getTrackSurroundOrientation(CallbackWithOneParam<Integer> callback);
483
484 /**
485  * 获取最小转弯半径
486  *
487  * @param callback 回调
488  */
489 void getMinRadius(CallbackWithOneParam<Float> callback);
490
491 /**
492  * 固定翼巡航速度
493  *
494  * @param callback 回调
495  */
496 void getMissionSpeed(CallbackWithOneParam<Float> callback);
497
498 /**
499  * 固定翼巡航速度
500  * @param value 速度
501  * @param callback 回调
502  */
503 void setMissionSpeed(float value, CallbackWithOneParam<Float> callback);
504
505 /**
506  * 获取最大升降速率
507  *

```

```

508     * @param callback 回调
509     */
510     void getMaxSpeed(CallbackWithOneParam<Float> callback);
511     /**
512     * 获取折叠机构的状态
513     *
514     * @param callback 回调
515     */
516     void getMechanismStatus(CallbackWithOneParam<FoldingMechanismStatus>
callback);
517
518     /**
519     * 执行机构配置参数设置
520     * @param type 机构类型
521     * @param callback 回调
522     */
523     void setActuatorParam(UavActuatorType type, CallbackWithNoParam callback);
524
525     /**
526     * 获取GNSS跟踪距离
527     * @param callback 回调
528     */
529     void getGnssTrackDistance(CallbackWithOneParam<Float> callback);
530
531     /**
532     * 获取GNSS跟踪半径
533     * @param callback 回调
534     */
535     void getGnssTrackRadius(CallbackWithOneParam<Float> callback);
536     /**
537     * 获取GNSS跟踪角度
538     * @param callback 回调
539     */
540     void getGnssTrackAngle(CallbackWithOneParam<Float> callback);
541
542     /**
543     * 获取GNSS跟踪完成动作
544     * @param callback 回调
545     */
546     void getGnssTrackFinishAction(CallbackWithOneParam<Integer> callback);
547
548     /**
549     * 获取GNSS返航航向
550     * @param callback 回调
551     */
552     void getGnssRtlYaw(CallbackWithOneParam<Float> callback);
553

```

```

554  /**
555   * 根据Cmd获取UAV参数
556   * @param cmdID CmdID
557   * @param callback 回调
558   */
559 void getUavParamById(String cmdID, CallbackWithOneParam<Integer> callback);
560
561 /**
562   * 设置ATTI状态参数
563   * @param value 0:正常 10:解禁 99:停桨
564   * @param callback 回调
565   */
566 void setAttiState(int value, CallbackWithOneParam<Integer> callback);
567 }

```

3.3.3 CruiserVisual接口

```

1  /**
2   * This interface is used to manage visual function in the aircraft, including
   avoidance and tracking.
3   */
4
5  public interface CruiserVisual extends AutelVisual {
6
7      /**
8       * 跟踪模式使能
9       *
10      * @param trackingAction 追踪模式
11      * @param callback 回调
12      */
13      void setTrackingMode(TrackingAction trackingAction, CallbackWithNoParam
callback);
14
15      /**
16       * 被检测到的目标信息上报
17       *
18      * @param callback the listener will be cancelled when the callback is
null.
19      */
20      void
setTrackingTargetListener(CallbackWithTwoParams<List<TrackingGoalArea>,
TrackingGoalArea> callback);
21
22      /**
23       * 追踪目标信息上报

```

```

24      *
25      * @param callback the listener will be cancelled when the callback is
null.
26      */
27      void setTrackingListener(CallbackWithOneParam<TrackingArea> callback);
28
29      /**
30      * 开始追踪
31      * @param mode 追踪模式
32      * @param callback 回调
33      */
34      @Deprecated
35      void startTracking(DynamicTrackMode mode, CallbackWithNoParam callback);
36
37      /**
38      * 设置追踪目标框
39      * @param trackingTarget 追踪目标
40      * @param callback 回调
41      */
42      void setTrackingGoalArea(TrackingTarget trackingTarget,
CallbackWithNoParam callback);
43
44      /**
45      * 设置激光照射状态监听
46      *
47      * @param listener the listener will be cancelled when the callback is
null.
48      */
49      void
setLaserShineStateInfoListeners(CallbackWithOneParam<LaserShineStateInfo>
listener);
50
51      /**
52      * 开始激光照射控制
53      *
54      * @param code 激光编码, 1-8 【默认填1】
55      * @param time 照射时长, 1-40 【默认填25】
56      * @param callback the listener will be cancelled when the callback is
null.
57      */
58      void setStartLaserShineCmd(int code, int time, CallbackWithNoParam
callback);
59
60      /**
61      * 开始激光测距控制
62      *
63      * @param mode 0单次测距, 1连续测距

```

```

64      * @param frequency 1或者5【默认选1】,当mode为1,即连续测距时有效,测距频率目前支持两个频率1HZ和5HZ
65      * @param callback the listener will be cancelled when the callback is null.
66      */
67      void setRangingLaserShineCmd(int mode, int frequency, CallbackWithNoParam callback);
68
69      /**
70       * 停止激光照射控制
71       *
72       * @param data0 预留,暂无意义
73       * @param callback the listener will be cancelled when the callback is null.
74       */
75      void setStopLaserShineCmd(int data0, CallbackWithNoParam callback);
76
77      /**
78       * 设置障碍物信息上报监听
79       *
80       * @param listener the listener will be cancelled when the callback is null.
81       */
82      void setObstacleCollisionAvoidListener(CallbackWithOneParam<ObstacleCollisionAvoid> listener);
83  }

```

3.3.4 CruiserGimbal接口

```

1  /**
2   * This interface is used to provide gimbal parameters and data for EVO.
3   */
4
5   public interface CruiserGimbal extends AutelGimbal {
6
7       /**
8        * Sets the listener for the gimbal pitch angle to monitor the real-time data.
9        * When set a null value, it will stop monitoring the gimbal angle data.
10       * The unit for returned integrate value is degree or °.
11       *
12       * @param listener the listener will be cancelled when the callback is null.
13       */
14       void setAngleListener(CallbackWithOneParam<EvoAngleInfo> listener);

```

```

14
15 /**
16  * 设置追踪天线角度监听
17  * @param listener the listener will be cancelled when the callback is
18  null.
19  */
20 void setTraceAntennaAngleListener(CallbackWithOneParam<AntennaAngleInfo>
21 listener);
22
23 /**
24  * 设置激光雷达数据监听
25  * @param listener the listener will be cancelled when the callback is
26  null.
27  */
28 void setLeaserRadarListener(CallbackWithOneParam<LeaserRadar> listener);
29
30 /**
31  * Sets the value of the gimbal pitch angle.
32  *
33  * @param pitchAngle the gimbal pitch angle, ranges from 0° to 90°.
34  */
35 void setGimbalAngle(int pitchAngle);
36
37 /**
38  * 设置云台角度
39  * @param pitch the pitch angle of the gimbal.
40  * @param yaw the yaw angle of the gimbal.
41  * @param roll the roll angle of the gimbal.
42  */
43 void setGimbalAngle(float pitch, float yaw, float roll);
44
45 /**
46  * 根据屏幕上的坐标点调整云台 指点居中
47  *
48  * @param x the x coordinate of the screen.
49  * @param y the y coordinate of the screen.
50  * @param pitch the pitch angle of the gimbal.
51  * @param yaw the yaw angle of the gimbal.
52  * @param roll the roll angle of the gimbal.
53  */
54 void adjustGimbalDirection(float x, float y, float pitch, float yaw, float
55 roll);
56
57 /**
58  * Sets an offset value of the gimbal pitch angle with a speed.
59  *
60  * @param speed the speed of the gimbal pitch.

```

```

57      * @param yaw the offset value of the gimbal yaw angle, ranges from -180°
to 180°.
58      */
59      void setGimbalAngleWithSpeed(int speed, int yaw);
60
61      /**
62       * Returns the parameter range manager of the gimbal.
63       *
64       * @return the parameter range manager of the gimbal.
65       */
66      EvoGimbalParameterRangeManager getParameterRangeManager();
67
68      /**
69       * Resets the gimbal angle.
70       *
71       * @param gimbalRoller the axis type of the gimbal.
72       * @param callback the callback when reset the gimbal angle.
73       */
74      void resetGimbalAngle(GimbalAxisType gimbalRoller, CallbackWithNoParam
callback);
75
76      /**
77       * Sets the roll axis adjustment of the gimbal.
78       *
79       * @param value the adjustment of the gimbal roll angle, ranges from
-10° to 10°.
80       */
81      void setRollAdjustData(float value);
82
83      /**
84       * Returns the roll angle of the gimbal.
85       *
86       * @param callback the callback when get the roll angle of the gimbal.
87       */
88      void getRollAdjustData(CallbackWithOneParam<Double> callback);
89
90      /**
91       * Sets the yaw axis adjustment of the gimbal.
92       *
93       * @param value the adjustment of the gimbal roll angle, ranges from
-10° to 10°.
94       */
95      void setYawAdjustData(float value);
96
97      /**
98       * Gets the yaw angle of the gimbal.
99       *

```



```

100     * @param callback the callback when get the roll angle of the gimbal.
101     */
102     void getYawAdjustData(CallbackWithOneParam<Double> callback);
103
104     /**
105     * Sets the pitch axis adjustment of the gimbal.
106     *
107     * @param value the adjustment of the gimbal roll angle, ranges from
108     *  $-10^{\circ}$  to  $10^{\circ}$ .
109     */
110     void setPitchAdjustData(float value);
111
112     /**
113     * Gets the pitch angle of the gimbal.
114     *
115     * @param callback the callback when get the roll angle of the gimbal.
116     */
117     void getPitchAdjustData(CallbackWithOneParam<Double> callback);
118
119     /**
120     * 待翻译。保存云台微调参数
121     *
122     * @param callback the callback when get the roll angle of the gimbal.
123     */
124     void setSaveParams(CallbackWithNoParam callback);
125
126     /**
127     * Converts to ReactiveX interface.
128     *
129     * @return ReactiveX interface.
130     */
131     RxCruiserGimbal toRx();
132
133     /**
134     * 待翻译：云台校准
135     *
136     * Sets gimbal calibration
137     *
138     * @param callback the callback when set gimbal calibration
139     */
140     void setGimbalCalibration(CallbackWithNoParam callback);
141
142     /**
143     * 待翻译：结束云台校准
144     *
145     * Sets gimbal calibration
146     *
147     * @param callback the callback when set gimbal calibration
148     */

```

```

146     void stopGimbalCalibration(CallbackWithNoParam callback);
147
148     /**
149      * Gets the roll angle of the gimbal.
150      *
151      * @param callback the callback when get the GimbalAdjustmentAngle of the
152      * gimbal.
153      */
154     void getAdjustGimbalAngelData(CallbackWithOneParam<GimbalAdjustmentAngle>
155     callback);
156
157     /**
158      * 设置目标位置Roll轴微调参数
159      * @param value the adjustment of the gimbal roll angle, ranges from
160      *  $-10^{\circ}$  to  $10^{\circ}$ .
161      */
162     void setTargetRollAdjustData(float value);
163
164     /**
165      * 设置目标位置Pitch微调参数
166      * @param value the adjustment of the gimbal pitch angle, ranges from
167      *  $-10^{\circ}$  to  $10^{\circ}$ .
168      */
169     void setTargetPitchAdjustData(float value);
170
171     /**
172      * 设置目标位置Yaw微调参数
173      * @param value the adjustment of the gimbal yaw angle, ranges from  $-10^{\circ}$  to
174      *  $10^{\circ}$ .
175      */
176     void setTargetYawAdjustData(float value);
177
178     /**
179      * 获取目标位置微调参数
180      * @param callback the callback when get the roll angle of the gimbal.
181      */
182     void getTargetGimbalAngelData(CallbackWithOneParam<GimbalAdjustmentAngle>
183     callback);
184
185     /**
186      * 待翻译。保存目标位置微调参数
187      *
188      * @param callback the callback when get the roll angle of the gimbal.
189      */
190     void setSaveTargetParams(CallbackWithNoParam callback);
191
192 }

```

3.3.4 LeaserRadar 激光雷达接口

```
1
2 public interface LeaserRadar {
3     /**
4      * 激光测距纬度坐标
5      */
6     double getLatitude();
7     /**
8      * 激光测距经度坐标
9      */
10    double getLongitude();
11    /**
12     * 激光测距高度坐标
13     */
14    float getAltitude();
15    /**
16     * 激光测距速度
17     */
18    float getSpeed();
19    /**
20     * 激光测距距离
21     */
22    float getDistance();
23    /**
24     * 激光测距角度
25     */
26    float getAngle();
27    /**
28     * 激光测距是否有效
29     */
30    boolean isValid();
31    /**
32     * 云台模组异常
33     */
34    boolean isGimbalLaserException();
35 }
36 }
```

4、新航线任务实现教程

4.1概述

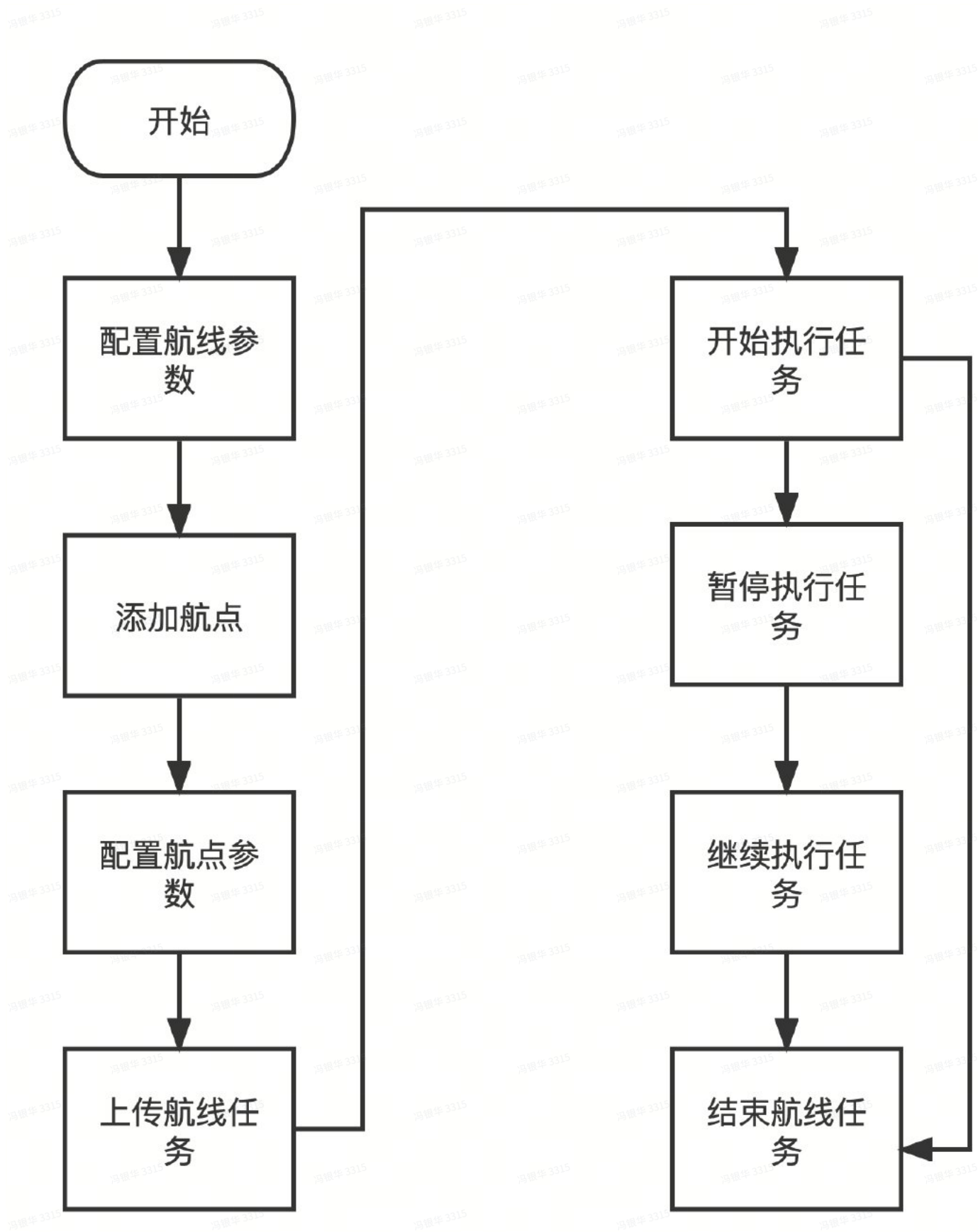
新航线任务支持用户手动选择目标点，通过不同的航线规划算法，自动生成飞行航线。用户可以自定义航线飞行参数以及航点的相机动作。算法库支持的任务有地面任务（包括航点任务、多边形任务、返航任务、降落任务、在线任务规划）

说明[航线算法规划文档](#)参考Demo工程里面文档说明

4.1.1流程图

为了使用任务规划功能，需要完成如下几个步骤

- 1、实例化航线任务，上传任务文件以及控制任务的开始、暂停，参看4.2.5节。
- 2、调用SDK算法生成航线任务文件，参看4.2.1节。



下面以Autel SDK demo为例讲解如何进行通过Autel SDK实现任务规划功能。示例以[DFWayPointActivity](#)类中参考

4.1.2航点任务说明

航点任务包含多个航点，当飞行器飞到对应的航点时，会执行预设动作，当动作完成以后，飞行器会飞到下一个航点执行预设动作，直到所有航点的任务完成。

4.1.3航点任务API说明

主要是PathPlanningParameter来设置航点任务，主要包含航线基本信息IntroInfo，降落信息LandInfo，任务信息MissionInfo信息，LaunchInfo进场降落航线的相关信息

航线算法返回结果在PathPlanningResult类中

PathPlanningParameter.java相关字段说明

```
1  /**
2   * 航线规划参数 基本信息
3   */
4  private IntroInfo introInfo;
5  /**
6   * 航线规划参数 进场及离场航线的相关信息
7   */
8  private LaunchInfo launchInfo;
9  /**
10   * 航线规划参数 进场降落航线的相关信息
11   */
12  private LandInfo landInfo;
13  /**
14   * 航线规划参数 强制降落航线的相关信息,暂时未使用
15   */
16  private ArrayList<ForceLandInfo> forceLandInfo;
17  /**
18   * 航线规划参数 任务航线的相关信息
19   */
20  private ArrayList<MissionInfo> missionInfo;
21  /**
22   * 算法返回的错误码
23   */
24  private ErrorCode errorCode;
```

IntroInfo.java相关说明

```
1  /**
2   * 按位取值进行表示该结构体的有效性:
3   * 1: LAUNCH;
4   * 2: LANDING;
5   * 3: FORCELAND;
6   * 4: MISSION;
```

```
7  * 二进制
8  */
9  private int planningType;
10 /**
11  * 作业任务部分有效的子作业任务的个数
12  */
13 private int subMisNum;
14 /**
15  * 有效的迫降点个数
16  */
17 private int forceLandNum;
18 /**
19  * 初始位置的纬度
20  */
21 private double initLat;
22 /**
23  * 初始位置的经度
24  */
25 private double initLon;
26 /**
27  * 初始位置的海拔高度
28  */
29 private float initAlt;
30 /**
31  * 最小转弯半径
32  */
33 private float MinRadius = 100;
34 private float MaxVz = 3;
35 /**
36  * 航线循环方式: -1: 无限循环; 0: 不循环; n: 循环n次;
37  */
38 private int CycleMode;
39 /**
40  * 航线循环起始子任务
41  */
42 private int StartSubID = 1;
43 /**
44  * 航线循环起始航点
45  */
46 private float StartWPID = 1;
47 /**
48  * 航线循环结束子任务
49  */
50 private int EndSubID = 1;
51 /**
52  * 航线循环结束航点
53  */
```

```
54 private float EndWPID = 1;
```

LandInfo.java相关说明

```
1 /**
2  * 按位取值进行表示该结构体的有效性:
3  * 1: LAUNCH;
4  * 2: LANDING;
5  * 3: FORCELAND;
6  * 4: MISSION;
7  * 二进制
8  */
9 private int planningType;
10 /**
11  * 作业任务部分有效的子作业任务的个数
12  */
13 private int subMisNum;
14 /**
15  * 有效的迫降点个数
16  */
17 private int forceLandNum;
18 /**
19  * 初始位置的纬度
20  */
21 private double initLat;
22 /**
23  * 初始位置的经度
24  */
25 private double initLon;
26 /**
27  * 初始位置的海拔高度
28  */
29 private float initAlt;
30 /**
31  * 最小转弯半径
32  */
33 private float MinRadius = 100;
34 private float MaxVz = 3;
35 /**
36  * 航线循环方式: -1: 无限循环; 0: 不循环; n: 循环n次;
37  */
38 private int CycleMode;
39 /**
40  * 航线循环起始子任务
41  */
```



```

42 private int StartSubID = 1;
43 /**
44  * 航线循环起始航点
45  */
46 private float StartWPID = 1;
47 /**
48  * 航线循环结束子任务
49  */
50 private int EndSubID = 1;
51 /**
52  * 航线循环结束航点
53  */
54 private float EndWPID = 1;

```

missionInfos信息说明

```

1 /**
2  * 任务结束动作: 1.LOITER; 2.GOHOME;
3  */
4 private int FinishMove;
5 /**
6  * 任务丢失动作: 1.LOITER; 2.GOHOME;
7  */
8 private int LinkLostMove;
9 /**
10 * 任务类型1: 自由航点航线 2: 区域测绘航线 3: 八字盘旋航线
11 */
12 private int subMissionType;
13 /**
14 * 子任务航线信息
15 */
16 private SubMissionInfo subMissionInfo;
17 /**
18 * 子任务航点信息
19 */
20 private ArrayList<WPInfo> wpInfo;
21 /**
22 * 子任务航点数量
23 */
24 private int WPNum;
25 /**
26 * 兴趣区域
27 */
28 private ArrayList<InterestArea> InterestArea;
29 /**

```

```
30 * 兴趣区域数量
31 */
32 private int IANum;
```

LaunchInfo.java信息说明

```
1 /**
2  * 起飞点纬度
3  */
4 private double launchLat;//
5 /**
6  * 起飞点经度
7  */
8 private double launchLon;//
9 /**
10 * 起飞点绝对高度（海拔高度）
11 */
12 private float launchAlt;//
13 /**
14 * 离场盘旋点纬度
15 */
16 private double departureLat;//
17 /**
18 * 离场盘旋点经度
19 */
20 private double departureLon;//
21 /**
22 * 离场盘旋点高度
23 */
24 private float departureAlt;//
25 /**
26 * 离场盘旋点高度类型 1：绝对高度（海拔）；2：相对高度；
27 */
28 private int altType;//
29 /**
30 * 离场盘旋半径
31 */
32 private float departureR;//
33 /**
34 * 离场飞行速度
35 */
36 private float departureVel;//
37 /**
38 * 垂直起飞相对高度
39 */
```

```
40 private float transAlt;//
```

ErrorCode.java错误码说明

```
1 /**
2  * 测绘区域多边形角点个数超限
3  */
4 CODE_10001(CodeType.NEW_ROUTE, 1),
5 /**
6  * 由普通航点生成的Mission Item个数超限
7  */
8 CODE_10002(CodeType.NEW_ROUTE, 2),
9 /**
10 * 由子任务生成的Mission Item个数超限
11 */
12 CODE_10003(CodeType.NEW_ROUTE, 4),
13 /**
14 * 构造降落模块时Mission Item个数超限
15 */
16 CODE_10004(CodeType.NEW_ROUTE, 8),
17 /**
18 * 由子任务生成的Mission Item个数超限
19 */
20 CODE_10005(CodeType.NEW_ROUTE, 16),
21 /**
22 * 构造降落模块时Mission Item个数超限
23 */
24 CODE_10006(CodeType.NEW_ROUTE, 32),
25 /**
26 * 不支持的任务类型PlanningType
27 */
28 CODE_10007(CodeType.NEW_ROUTE, 64),
29 /**
30 * 不支持的子任务类型SubMissionType
31 */
32 CODE_10008(CodeType.NEW_ROUTE, 128),
33 /**
34 * 二维片段twoD_block个数超限
35 */
36 CODE_10009(CodeType.NEW_ROUTE, 256),
37 /**
38 * 三维片段threeD_block个数超限
39 */
40 CODE_10010(CodeType.NEW_ROUTE, 512),
41 /**
```

```
42 * SampleNum航线采样点个数超限 (航线超过100公里)
43 */
44 CODE_10011(CodeType.NEW_ROUTE, 1024),
45 /**
46 * 前后两个采样点的经度纬度一模一样
47 */
48 CODE_10012(CodeType.NEW_ROUTE, 2048),
49 /**
50 * 相邻两点无效的组合, 如返航落地后再加一个航点, 或者无线盘旋后再加一个航点
51 */
52 CODE_10013(CodeType.NEW_ROUTE, 4096),
53 /**
54 * 观察动线的锚点数量小于2
55 */
56 CODE_20001(CodeType.INSPECTION, 1),
57 /**
58 * 观察动线的锚点数量大于1000
59 */
60 CODE_20002(CodeType.INSPECTION, 2),
61 /**
62 * 航带宽度较宽且大幅弯折, 生成的航点数量比锚点少, 不能一一对应
63 */
64 CODE_20003(CodeType.INSPECTION, 4),
65 /**
66 * 观察动线反向转折, 无法生成航线
67 */
68 CODE_20004(CodeType.INSPECTION, 8),
69 /**
70 * 对航线采样查地形时, 采样点数量大于10000。采样间隔10米, 共100公里
71 */
72 CODE_20005(CodeType.INSPECTION, 16),
73 /**
74 * 算法内部的航点分组超过1000个
75 */
76 CODE_20006(CodeType.INSPECTION, 32),
77 /**
78 * 算法分组内部的航点个数超过100个
79 */
80 CODE_20007(CodeType.INSPECTION, 64),
81 /**
82 * 非法的相机视场角, 负数或超过180度
83 */
84 CODE_20008(CodeType.INSPECTION, 128),
85 /**
86 * 目标幅宽多边形生成失败, 因为有环形
87 */
88 CODE_20009(CodeType.INSPECTION, 256),
```

```

89  /**
90   * 折线偏移无法计算, 航线生成失败
91   */
92   CODE_20010(CodeType.INSPECTION, 512),
93
94  /**
95   * home点坐标或上下盘旋点非法
96   */
97   CODE_90001(CodeType.COMMON, 90001),
98  /**
99   * 航点任务为null
100  */
101  CODE_90002(CodeType.COMMON, 90002),
102  /**
103   * 矩形或多边形任务为null
104   */
105  CODE_90003(CodeType.COMMON, 90003),
106  /**
107   * 巡检任务为null
108   */
109  CODE_90004(CodeType.COMMON, 90004),
110
111  UNKNOWN(CodeType.UNKNOWN, -1);

```

PathPlanningResult.java类说明

```

1
2  public class PathPlanningResult {
3
4      /**
5       * 采样点个数
6       */
7      private int sampleNum;
8      /**
9       * 采样点纬度
10     */
11     private double[] sampleLat;
12     /**
13      * 采样点经度
14      */
15     private double[] sampleLon;
16     /**
17      * 采样点海拔高度
18      */
19     private float[] sampleAlt;

```

```
20
21 //显示航点，用于航点切换，与正常航点序号区分
22 /**
23  * 显示航点个数
24  */
25 private int displayNum;
26 /**
27  * 显示航点ID
28  */
29 private char[] displayID;
30 /**
31  * 显示航点纬度
32  */
33 private double[] displayLat;
34 /**
35  * 显示航点经度
36  */
37 private double[] displayLon;
38 /**
39  * 显示航点海拔
40  */
41 private float[] displayAlt;
42
43
44 /**
45  * 迫降点
46  * 迫降采样点个数
47  */
48 private int fLandNum;
49 /**
50  * 迫降采样点纬度
51  */
52 private double[] fLandLat;
53 /**
54  * 迫降采样点经度
55  */
56 private double[] fLandLon;
57 /**
58  * 迫降采样点海拔
59  */
60 private float[] fLandAlt;
61
62 /**
63  * 加号个数
64  */
65 private int plusNumber;
66 /**
```

```

67      * 加号纬度
68      */
69      private double[] plusLat;
70      /**
71      * 加号经度
72      */
73      private double[] plusLon;
74      /**
75      * 加号海拔高度
76      */
77      private float[] plusAlt;
78      /**
79      * 加号位于哪个航点
80      */
81      private char[] plusWP_id;
82
83      /**
84      * 箭头个数
85      */
86      private int arrowNumber;
87      /**
88      * 箭头起点纬度
89      */
90      private double[] arrowFromLat;
91      /**
92      * 箭头起点经度
93      */
94      private double[] arrowFromLon;
95      /**
96      * 箭头起点高度
97      */
98      private float[] arrowFromAlt;
99      /**
100     * 箭头终点纬度
101     */
102     private double[] arrowToLat;
103     /**
104     * 箭头终点经度
105     */
106     private double[] arrowToLon;
107     /**
108     * 箭头终点海拔高度
109     */
110     private float[] arrowToAlt;
111     /**
112     * 箭头指向 (单位为度, 正北为0, 顺时针到360度)
113     */

```

```
114     private float[] arrowHeading;
115
116     /**
117      * 拍照点个数
118      */
119     private int photoNum;
120
121     /**
122      * 拍照点纬度
123      */
124     private double[] photoLat;
125
126     /**
127      * 拍照点经度
128      */
129     private double[] photoLon;
130
131     /**
132      * 拍照点海拔高度
133      */
134     private float[] photoAlt;
135
136     /**
137      * 标签个数
138      */
139     private int altitudeNumber;
140
141     /**
142      * 1~wpNum: 第1到wpNum号航点
143      * 标签类型
144      * 0: 垂直起飞到达点;
145      * -1: 上升表征点;
146      * -2: 上升盘旋点;
147      * -3: 下降盘旋点;
148      * -4: 下降表征点;
149      * -5: 垂直降落开始点;
150      */
151     private short[] altitudeType;
152
153     /**
154      * 标签纬度
155      */
156     private double[] altitudeLat;
157
158     /**
159      * 标签经度
160      */
161     private double[] altitudeLon;
162
163     /**
164      * 航点海拔高度 (也是标签高度)
165      */
166     private float[] altitudeAlt;
```



```

161  /**
162   * 标签的航程（从起飞点算起）
163   */
164   private float[] altitudeLength;
165  /**
166   * 标签对应采样的序号
167   */
168   private char[] altitudeSampleIndex;
169
170  /**
171   * 测绘区域面积和高度
172   * 测绘区域个数
173   */
174   private int surveyZoneNumber;
175  /**
176   * 中心纬度
177   */
178   private double[] surveyZoneLat;
179  /**
180   * 中心经度
181   */
182   private double[] surveyZoneLon;
183  /**
184   * 海拔高度
185   */
186   private float[] surveyZoneAlt;
187  /**
188   * 相对高度
189   */
190   private float[] surveyZoneRelative_H;
191  /**
192   * 测绘区域面积
193   */
194   private float[] surveyZoneArea;
195  /**
196   * 主航线推荐角度
197   */
198   private float[] recommendDirection;
199
200
201  /**
202   * 位置 二进制表示      错误原因
203   * 1    000000000001    测绘区域多边形角点个数超限
204   * 2    000000000010    由普通航点生成的Mission Item个数超限
205   * 3    000000000100    由测绘区域生成的Mission Item个数超限
206   * 4    000000001000    构造起飞模块时Mission Item个数超限
207   * 5    000000010000    由子任务生成的Mission Item个数超限

```

```

208      * 6    000001000000    构造降落模块时Mission Item个数超限
209      * 7    000010000000    不支持的任务类型PlanningType
210      * 8    000100000000    不支持的子任务类型SubMissionType
211      * 9    001000000000    二维片段twoD_block个数超限
212      * 10   010000000000    三维片段threeD_block个数超限
213      * 11   100000000000    SampleNum航线采样点个数超限
214      * ErrCode错误码的意义，错误码转换成二进制，如果等于0则没有错误，如果不为0则发生了错误，二进制的每一位代表的一个错误原因。
215      */
216      private int errorCode;
217      /**
218       * 航线总耗时
219       */
220      private float totalTime;
221      /**
222       * 航线总长度
223       */
224      private float totalLength;
225      /**
226       * 调用算法之前报错码
227       */
228      private ErrorCode otherError;
229  }

```

4.2示例

4.2.1航线算法规划

```

1  //测试数据 6个航点 前四个航点分别设置了相机动作定时拍照，定距拍照，录像，后三个都是无动作，第五个航点设置转弯模式为定圈盘旋，第六个航点设置转弯模式为定时盘旋
2  //具体字段含义见工程目录下面"航线规划算法文档" <<“龙鱼”在线规划输入输出定义V6_8月15日>>
3  String waypointMission = "{\n" +
4      "\t\"forceLandInfo\": [],\n" +
5      "\t\"introInfo\": {\n" +
6      "\t\t\"CycleMode\": 0,\n" +
7      "\t\t\"EndSubID\": 1,\n" +
8      "\t\t\"EndWPID\": 1.0,\n" +
9      "\t\t\"MaxVz\": 3.0,\n" +
10     "\t\t\"MinRadius\": 200.0,\n" +
11     "\t\t\"StartSubID\": 1,\n" +
12     "\t\t\"StartWPID\": 1.0,\n" +
13     "\t\t\"forceLandNum\": 0,\n" +
14     "\t\t\"initAlt\": 0.0,\n" +

```

```
15  "\\t\\t\\\"initLat\\\": 22.51400425525294,\\n\" +
16  "\\t\\t\\\"initLon\\\": 113.92571262532796,\\n\" +
17  "\\t\\t\\\"planningType\\\": 11,\\n\" +
18  "\\t\\t\\\"subMisNum\\\": 1\\n\" +
19  "\\t},\\n\" +
20  "\\t\\\"landInfo\\\": {\\n\" +
21  "\\t\\t\\\"altType\\\": 2,\\n\" +
22  "\\t\\t\\\"approachAlt\\\": 100.0,\\n\" +
23  "\\t\\t\\\"approachLat\\\": 22.514004534760666,\\n\" +
24  "\\t\\t\\\"approachLon\\\": 113.93263822751037,\\n\" +
25  "\\t\\t\\\"approachR\\\": 200.0,\\n\" +
26  "\\t\\t\\\"approachVel\\\": 20.0,\\n\" +
27  "\\t\\t\\\"homeAlt\\\": 0.0,\\n\" +
28  "\\t\\t\\\"homeAltType\\\": 2,\\n\" +
29  "\\t\\t\\\"homeLat\\\": 22.51400425525294,\\n\" +
30  "\\t\\t\\\"homeLon\\\": 113.92571262532796,\\n\" +
31  "\\t\\t\\\"transAlt\\\": 40.0\\n\" +
32  "\\t},\\n\" +
33  "\\t\\\"launchInfo\\\": {\\n\" +
34  "\\t\\t\\\"altType\\\": 2,\\n\" +
35  "\\t\\t\\\"departureAlt\\\": 100.0,\\n\" +
36  "\\t\\t\\\"departureLat\\\": 22.514004534760666,\\n\" +
37  "\\t\\t\\\"departureLon\\\": 113.91878702314557,\\n\" +
38  "\\t\\t\\\"departureR\\\": 200.0,\\n\" +
39  "\\t\\t\\\"departureVel\\\": 20.0,\\n\" +
40  "\\t\\t\\\"launchAlt\\\": 0.0,\\n\" +
41  "\\t\\t\\\"launchLat\\\": 22.51400425525294,\\n\" +
42  "\\t\\t\\\"launchLon\\\": 113.92571262532796,\\n\" +
43  "\\t\\t\\\"transAlt\\\": 40.0\\n\" +
44  "\\t},\\n\" +
45  "\\t\\\"missionInfo\\\": [{\\n\" +
46  "\\t\\t\\\"FinishMove\\\": 1,\\n\" +
47  "\\t\\t\\\"IANum\\\": 0,\\n\" +
48  "\\t\\t\\\"InterestArea\\\": [],\\n\" +
49  "\\t\\t\\\"LinkLostMove\\\": 2,\\n\" +
50  "\\t\\t\\\"WPNum\\\": 6,\\n\" +
51  "\\t\\t\\\"subMissionInfo\\\": {\\n\" +
52  "\\t\\t\\t\\\"airLineDir\\\": 0.0,\\n\" +
53  "\\t\\t\\t\\\"baseAlt\\\": 0.0,\\n\" +
54  "\\t\\t\\t\\\"focalLength\\\": 25.6,\\n\" +
55  "\\t\\t\\t\\\"overlapAlong\\\": 0,\\n\" +
56  "\\t\\t\\t\\\"overlapSide\\\": 0,\\n\" +
57  "\\t\\t\\t\\\"pixNumX\\\": 22489,\\n\" +
58  "\\t\\t\\t\\\"pixNumY\\\": 12637,\\n\" +
59  "\\t\\t\\t\\\"resolution\\\": 0.0,\\n\" +
60  "\\t\\t\\t\\\"sensorOrient\\\": 1,\\n\" +
61  "\\t\\t\\t\\\"sensorSizeX\\\": 35.983963,\\n\" +
```

```
62     "\t\t\t\"sensorSizeY\": 20.219847\n" +
63     "\t\t},\n" +
64     "\t\t\"subMissionType\": 1,\n" +
65     "\t\t\"wpInfo\": [{\n" +
66     "\t\t\t\t\"actionParam1\": 2.0,\n" +
67     "\t\t\t\t\"gimbalPitch\": -42.0,\n" +
68     "\t\t\t\t\"gimbalYaw\": 71.0,\n" +
69     "\t\t\t\t\"payloadAction\": 2,\n" +
70     "\t\t\t\t\"wpAlt\": 100.0,\n" +
71     "\t\t\t\t\"wpAltType\": 2,\n" +
72     "\t\t\t\t\"wpClimbMode\": 1,\n" +
73     "\t\t\t\t\"wpIndex\": 1,\n" +
74     "\t\t\t\t\"wpLat\": 22.517180545105873,\n" +
75     "\t\t\t\t\"wpLon\": 113.92888891518089,\n" +
76     "\t\t\t\t\"wpRadius\": 200.0,\n" +
77     "\t\t\t\t\"wpReserved1\": 0.0,\n" +
78     "\t\t\t\t\"wpTurnMode\": 2,\n" +
79     "\t\t\t\t\"wpTurnParam1\": 1.0,\n" +
80     "\t\t\t\t\"wpType\": 4,\n" +
81     "\t\t\t\t\"wpVel\": 20.0\n" +
82     "\t\t\t}, {\n" +
83     "\t\t\t\t\"actionParam1\": 30.0,\n" +
84     "\t\t\t\t\"gimbalPitch\": -31.0,\n" +
85     "\t\t\t\t\"gimbalYaw\": -59.0,\n" +
86     "\t\t\t\t\"payloadAction\": 3,\n" +
87     "\t\t\t\t\"wpAlt\": 100.0,\n" +
88     "\t\t\t\t\"wpAltType\": 2,\n" +
89     "\t\t\t\t\"wpClimbMode\": 1,\n" +
90     "\t\t\t\t\"wpIndex\": 2,\n" +
91     "\t\t\t\t\"wpLat\": 22.51922217962033,\n" +
92     "\t\t\t\t\"wpLon\": 113.92471539799595,\n" +
93     "\t\t\t\t\"wpRadius\": 200.0,\n" +
94     "\t\t\t\t\"wpReserved1\": 0.0,\n" +
95     "\t\t\t\t\"wpTurnMode\": 2,\n" +
96     "\t\t\t\t\"wpTurnParam1\": 1.0,\n" +
97     "\t\t\t\t\"wpType\": 4,\n" +
98     "\t\t\t\t\"wpVel\": 20.0\n" +
99     "\t\t\t}, {\n" +
100     "\t\t\t\t\"actionParam1\": 0.0,\n" +
101     "\t\t\t\t\"gimbalPitch\": -38.0,\n" +
102     "\t\t\t\t\"gimbalYaw\": 96.0,\n" +
103     "\t\t\t\t\"payloadAction\": 4,\n" +
104     "\t\t\t\t\"wpAlt\": 100.0,\n" +
105     "\t\t\t\t\"wpAltType\": 2,\n" +
106     "\t\t\t\t\"wpClimbMode\": 1,\n" +
107     "\t\t\t\t\"wpIndex\": 3,\n" +
108     "\t\t\t\t\"wpLat\": 22.519885273960526,\n" +
```

109 "\\t\\t\\t\\wpLon\\": 113.93204621923553,\\n" +
110 "\\t\\t\\t\\wpRadius\\": 200.0,\\n" +
111 "\\t\\t\\t\\wpReserved1\\": 0.0,\\n" +
112 "\\t\\t\\t\\wpTurnMode\\": 2,\\n" +
113 "\\t\\t\\t\\wpTurnParam1\\": 1.0,\\n" +
114 "\\t\\t\\t\\wpType\\": 4,\\n" +
115 "\\t\\t\\t\\wpVel\\": 20.0\\n" +
116 "\\t\\t\\t\\", {\\n" +
117 "\\t\\t\\t\\actionParam1\\": 0.0,\\n" +
118 "\\t\\t\\t\\gimbalPitch\\": -42.0,\\n" +
119 "\\t\\t\\t\\gimbalYaw\\": 20.0,\\n" +
120 "\\t\\t\\t\\payloadAction\\": 0,\\n" +
121 "\\t\\t\\t\\wpAlt\\": 100.0,\\n" +
122 "\\t\\t\\t\\wpAltType\\": 2,\\n" +
123 "\\t\\t\\t\\wpClimbMode\\": 1,\\n" +
124 "\\t\\t\\t\\wpIndex\\": 4,\\n" +
125 "\\t\\t\\t\\wpLat\\": 22.516633741529645,\\n" +
126 "\\t\\t\\t\\wpLon\\": 113.93664236660521,\\n" +
127 "\\t\\t\\t\\wpRadius\\": 200.0,\\n" +
128 "\\t\\t\\t\\wpReserved1\\": 0.0,\\n" +
129 "\\t\\t\\t\\wpTurnMode\\": 2,\\n" +
130 "\\t\\t\\t\\wpTurnParam1\\": 1.0,\\n" +
131 "\\t\\t\\t\\wpType\\": 4,\\n" +
132 "\\t\\t\\t\\wpVel\\": 20.0\\n" +
133 "\\t\\t\\t\\", {\\n" +
134 "\\t\\t\\t\\actionParam1\\": 0.0,\\n" +
135 "\\t\\t\\t\\gimbalPitch\\": -42.0,\\n" +
136 "\\t\\t\\t\\gimbalYaw\\": 20.0,\\n" +
137 "\\t\\t\\t\\payloadAction\\": 0,\\n" +
138 "\\t\\t\\t\\wpAlt\\": 100.0,\\n" +
139 "\\t\\t\\t\\wpAltType\\": 2,\\n" +
140 "\\t\\t\\t\\wpClimbMode\\": 1,\\n" +
141 "\\t\\t\\t\\wpIndex\\": 5,\\n" +
142 "\\t\\t\\t\\wpLat\\": 22.513700066361963,\\n" +
143 "\\t\\t\\t\\wpLon\\": 113.9360415517898,\\n" +
144 "\\t\\t\\t\\wpRadius\\": 200.0,\\n" +
145 "\\t\\t\\t\\wpReserved1\\": 0.0,\\n" +
146 "\\t\\t\\t\\wpTurnMode\\": 4,\\n" +
147 "\\t\\t\\t\\wpTurnParam1\\": 2.0,\\n" +
148 "\\t\\t\\t\\wpType\\": 4,\\n" +
149 "\\t\\t\\t\\wpVel\\": 20.0\\n" +
150 "\\t\\t\\t\\", {\\n" +
151 "\\t\\t\\t\\actionParam1\\": 0.0,\\n" +
152 "\\t\\t\\t\\gimbalPitch\\": -42.0,\\n" +
153 "\\t\\t\\t\\gimbalYaw\\": 20.0,\\n" +
154 "\\t\\t\\t\\payloadAction\\": 0,\\n" +
155 "\\t\\t\\t\\wpAlt\\": 100.0,\\n" +


```
23  "\\t\\t\\\"approachLat\\\": 22.597084893911017,\\n\" +
24  "\\t\\t\\\"approachLon\\\": 114.0587747424179,\\n\" +
25  "\\t\\t\\\"approachR\\\": 200.0,\\n\" +
26  "\\t\\t\\\"approachVel\\\": 19.820269,\\n\" +
27  "\\t\\t\\\"homeAlt\\\": 0.0,\\n\" +
28  "\\t\\t\\\"homeAltType\\\": 2,\\n\" +
29  "\\t\\t\\\"homeLat\\\": 22.597084613515303,\\n\" +
30  "\\t\\t\\\"homeLon\\\": 114.05184496791044,\\n\" +
31  "\\t\\t\\\"transAlt\\\": 40.0\\n\" +
32  "\\t},\\n\" +
33  "\\t\\\"launchInfo\\\": {\\n\" +
34  "\\t\\t\\\"altType\\\": 1,\\n\" +
35  "\\t\\t\\\"departureAlt\\\": 820.0,\\n\" +
36  "\\t\\t\\\"departureLat\\\": 22.597084893911017,\\n\" +
37  "\\t\\t\\\"departureLon\\\": 114.04491519340303,\\n\" +
38  "\\t\\t\\\"departureR\\\": 200.0,\\n\" +
39  "\\t\\t\\\"departureVel\\\": 19.820269,\\n\" +
40  "\\t\\t\\\"launchAlt\\\": 0.0,\\n\" +
41  "\\t\\t\\\"launchLat\\\": 22.597084613515303,\\n\" +
42  "\\t\\t\\\"launchLon\\\": 114.05184496791044,\\n\" +
43  "\\t\\t\\\"transAlt\\\": 40.0\\n\" +
44  "\\t},\\n\" +
45  "\\t\\\"missionInfo\\\": [{\\n\" +
46  "\\t\\t\\\"FinishMove\\\": 1,\\n\" +
47  "\\t\\t\\\"IANum\\\": 0,\\n\" +
48  "\\t\\t\\\"InterestArea\\\": [],\\n\" +
49  "\\t\\t\\\"LinkLostMove\\\": 2,\\n\" +
50  "\\t\\t\\\"WPNum\\\": 4,\\n\" +
51  "\\t\\t\\\"subMissionInfo\\\": {\\n\" +
52  "\\t\\t\\t\\\"airLineDir\\\": 10.0,\\n\" +
53  "\\t\\t\\t\\\"baseAlt\\\": 20.0,\\n\" +
54  "\\t\\t\\t\\\"focalLength\\\": 25.6,\\n\" +
55  "\\t\\t\\t\\\"overlapAlong\\\": 80,\\n\" +
56  "\\t\\t\\t\\\"overlapSide\\\": 70,\\n\" +
57  "\\t\\t\\t\\\"pixNumX\\\": 22489,\\n\" +
58  "\\t\\t\\t\\\"pixNumY\\\": 12637,\\n\" +
59  "\\t\\t\\t\\\"resolution\\\": 5.0,\\n\" +
60  "\\t\\t\\t\\\"sensorOrient\\\": 1,\\n\" +
61  "\\t\\t\\t\\\"sensorSizeX\\\": 35.983963,\\n\" +
62  "\\t\\t\\t\\\"sensorSizeY\\\": 20.219847\\n\" +
63  "\\t\\t},\\n\" +
64  "\\t\\t\\\"subMissionType\\\": 2,\\n\" +
65  "\\t\\t\\\"wpInfo\\\": [{\\n\" +
66  "\\t\\t\\t\\\"actionParam1\\\": 0.0,\\n\" +
67  "\\t\\t\\t\\\"gimbalPitch\\\": -90.0,\\n\" +
68  "\\t\\t\\t\\\"gimbalYaw\\\": 0.0,\\n\" +
69  "\\t\\t\\t\\\"payloadAction\\\": 1,\\n\" +
```

```
70  "\\t\\t\\t\\wpAlt\\": 800.000006,\\n" +
71  "\\t\\t\\t\\wpAltType\\": 2,\\n" +
72  "\\t\\t\\t\\wpClimbMode\\": 1,\\n" +
73  "\\t\\t\\t\\wpIndex\\": 1,\\n" +
74  "\\t\\t\\t\\wpLat\\": 22.60661348307411,\\n" +
75  "\\t\\t\\t\\wpLon\\": 114.0486686780575,\\n" +
76  "\\t\\t\\t\\wpRadius\\": 200.0,\\n" +
77  "\\t\\t\\t\\wpReserved1\\": 0.0,\\n" +
78  "\\t\\t\\t\\wpTurnMode\\": 2,\\n" +
79  "\\t\\t\\t\\wpTurnParam1\\": 0.0,\\n" +
80  "\\t\\t\\t\\wpType\\": 5,\\n" +
81  "\\t\\t\\t\\wpVel\\": 19.820269\\n" +
82  "\\t\\t\\t}, {\\n" +
83  "\\t\\t\\t\\actionParam1\\": 0.0,\\n" +
84  "\\t\\t\\t\\gimbalPitch\\": -90.0,\\n" +
85  "\\t\\t\\t\\gimbalYaw\\": 0.0,\\n" +
86  "\\t\\t\\t\\payloadAction\\": 1,\\n" +
87  "\\t\\t\\t\\wpAlt\\": 800.000006,\\n" +
88  "\\t\\t\\t\\wpAltType\\": 2,\\n" +
89  "\\t\\t\\t\\wpClimbMode\\": 1,\\n" +
90  "\\t\\t\\t\\wpIndex\\": 2,\\n" +
91  "\\t\\t\\t\\wpLat\\": 22.60661348307411,\\n" +
92  "\\t\\t\\t\\wpLon\\": 114.06137383746925,\\n" +
93  "\\t\\t\\t\\wpRadius\\": 200.0,\\n" +
94  "\\t\\t\\t\\wpReserved1\\": 0.0,\\n" +
95  "\\t\\t\\t\\wpTurnMode\\": 2,\\n" +
96  "\\t\\t\\t\\wpTurnParam1\\": 0.0,\\n" +
97  "\\t\\t\\t\\wpType\\": 5,\\n" +
98  "\\t\\t\\t\\wpVel\\": 19.820269\\n" +
99  "\\t\\t\\t}, {\\n" +
100 "\\t\\t\\t\\actionParam1\\": 0.0,\\n" +
101 "\\t\\t\\t\\gimbalPitch\\": -90.0,\\n" +
102 "\\t\\t\\t\\gimbalYaw\\": 0.0,\\n" +
103 "\\t\\t\\t\\payloadAction\\": 1,\\n" +
104 "\\t\\t\\t\\wpAlt\\": 800.000006,\\n" +
105 "\\t\\t\\t\\wpAltType\\": 2,\\n" +
106 "\\t\\t\\t\\wpClimbMode\\": 1,\\n" +
107 "\\t\\t\\t\\wpIndex\\": 3,\\n" +
108 "\\t\\t\\t\\wpLat\\": 22.593908323662365,\\n" +
109 "\\t\\t\\t\\wpLon\\": 114.06137383746925,\\n" +
110 "\\t\\t\\t\\wpRadius\\": 200.0,\\n" +
111 "\\t\\t\\t\\wpReserved1\\": 0.0,\\n" +
112 "\\t\\t\\t\\wpTurnMode\\": 2,\\n" +
113 "\\t\\t\\t\\wpTurnParam1\\": 0.0,\\n" +
114 "\\t\\t\\t\\wpType\\": 5,\\n" +
115 "\\t\\t\\t\\wpVel\\": 19.820269\\n" +
116 "\\t\\t\\t}, {\\n" +
```



```

117     "\t\t\t\"actionParam1\": 0.0,\n" +
118     "\t\t\t\"gimbalPitch\": -90.0,\n" +
119     "\t\t\t\"gimbalYaw\": 0.0,\n" +
120     "\t\t\t\"payloadAction\": 1,\n" +
121     "\t\t\t\"wpAlt\": 800.00006,\n" +
122     "\t\t\t\"wpAltType\": 2,\n" +
123     "\t\t\t\"wpClimbMode\": 1,\n" +
124     "\t\t\t\"wpIndex\": 4,\n" +
125     "\t\t\t\"wpLat\": 22.593908323662365,\n" +
126     "\t\t\t\"wpLon\": 114.0486686780575,\n" +
127     "\t\t\t\"wpRadius\": 200.0,\n" +
128     "\t\t\t\"wpReserved1\": 0.0,\n" +
129     "\t\t\t\"wpTurnMode\": 2,\n" +
130     "\t\t\t\"wpTurnParam1\": 0.0,\n" +
131     "\t\t\t\"wpType\": 5,\n" +
132     "\t\t\t\"wpVel\": 19.820269\n" +
133     "\t\t}]\n" +
134     "\t}]\n" +
135     "}";
136 PathPlanningResult pathPlanningResult =
    RealTimePathPlaningUtils.getMissionPath(mappingJson);
137 Log.d(TAG, "testMapping: pathPlanningResult -> " + pathPlanningResult);
138 Toast.makeText(this, "testMapping: pathPlanningResult -> " +
    pathPlanningResult, Toast.LENGTH_SHORT).show();

```

4.3.3降落任务数据示例

```

1 String onLineJson = "{\n" +
2     "\t\"forceLandInfo\": [],\n" +
3     "\t\"introInfo\": {\n" +
4     "\t\t\"CycleMode\": 0,\n" +
5     "\t\t\"EndSubID\": 1,\n" +
6     "\t\t\"EndWPID\": 1.0,\n" +
7     "\t\t\"MaxVz\": 3.0,\n" +
8     "\t\t\"MinRadius\": 138.0,\n" +
9     "\t\t\"StartSubID\": 1,\n" +
10    "\t\t\"StartWPID\": 1.0,\n" +
11    "\t\t\"forceLandNum\": 0,\n" +
12    "\t\t\"initAlt\": 10.0,\n" +
13    "\t\t\"initLat\": 22.601954185582244,\n" +
14    "\t\t\"initLon\": 114.00908924274069,\n" +
15    "\t\t\"planningType\": 2,\n" +
16    "\t\t\"subMisNum\": 0\n" +
17    "\t},\n" +
18    "\t\"landInfo\": {\n" +

```

```

19     "\t\t\"altType\": 1,\n" +
20     "\t\t\"approachAlt\": 100.0,\n" +
21     "\t\t\"approachLat\": 22.595354103352392,\n" +
22     "\t\t\"approachLon\": 114.00598790081068,\n" +
23     "\t\t\"approachR\": 138.0,\n" +
24     "\t\t\"approachVel\": 20.0,\n" +
25     "\t\t\"homeAlt\": 10.0,\n" +
26     "\t\t\"homeAltType\": 2,\n" +
27     "\t\t\"homeLat\": 22.601954185582244,\n" +
28     "\t\t\"homeLon\": 114.00908924274069,\n" +
29     "\t\t\"transAlt\": 100.0\n" +
30     "\t},\n" +
31     "\t\"launchInfo\": {\n" +
32     "\t\t\"altType\": 0,\n" +
33     "\t\t\"departureAlt\": 0.0,\n" +
34     "\t\t\"departureLat\": 0.0,\n" +
35     "\t\t\"departureLon\": 0.0,\n" +
36     "\t\t\"departureR\": 0.0,\n" +
37     "\t\t\"departureVel\": 0.0,\n" +
38     "\t\t\"launchAlt\": 0.0,\n" +
39     "\t\t\"launchLat\": 0.0,\n" +
40     "\t\t\"launchLon\": 0.0,\n" +
41     "\t\t\"transAlt\": 0.0\n" +
42     "\t},\n" +
43     "\t\"missionInfo\": []\n" +
44     "}";
45 PathPlanningResult pathPlanningResult =
46     RealTimePathPlaningUtils.getMissionPath(onLineJson);
47 Log.d(TAG, "onLineJson: pathPlanningResult -> " + pathPlanningResult);
48 Toast.makeText(this, "onLineJson: pathPlanningResult -> " +
49     pathPlanningResult, Toast.LENGTH_SHORT).show();

```

其他类型的任务算法调用示例，雷同 根据具体的数据结构及类型决定

4.2.4规划航线写入本地

```

1 /**
2  * 写入航线文件
3  *
4  * @param filePath 文件路径
5  * @param pathPlanningParameter 航线规划参数
6  * @return 0:成功 1:失败
7  */

```

```
8 public static native int writeNewMissionFile(String filePath,  
PathPlanningParameter pathPlanningParameter);
```

示例如下:

```
1 File myDir = new File(FileUtils.getMissionFilePath());  
2 if (!myDir.exists()) {  
3     myDir.mkdirs();  
4 }  
5 //转换成json字符串 可通过taskModel对象 转换调用getPathPlanningParameter(taskModel,  
droneLocation, subMissionInfo);方法  
6  
7 String waypointMission = "{\n" +  
8     "\t\"forceLandInfo\": [],\n" +  
9     "\t\"introInfo\": {\n" +  
10        "\t\t\"CycleMode\": 0,\n" +  
11        "\t\t\"EndSubID\": 1,\n" +  
12        "\t\t\"EndWPID\": 1.0,\n" +  
13        "\t\t\"MaxVz\": 3.0,\n" +  
14        "\t\t\"MinRadius\": 200.0,\n" +  
15        "\t\t\"StartSubID\": 1,\n" +  
16        "\t\t\"StartWPID\": 1.0,\n" +  
17        "\t\t\"forceLandNum\": 0,\n" +  
18        "\t\t\"initAlt\": 0.0,\n" +  
19        "\t\t\"initLat\": 22.51400425525294,\n" +  
20        "\t\t\"initLon\": 113.92571262532796,\n" +  
21        "\t\t\"planningType\": 11,\n" +  
22        "\t\t\"subMisNum\": 1\n" +  
23        "\t},\n" +  
24        "\t\"landInfo\": {\n" +  
25        "\t\t\"altType\": 2,\n" +  
26        "\t\t\"approachAlt\": 100.0,\n" +  
27        "\t\t\"approachLat\": 22.514004534760666,\n" +  
28        "\t\t\"approachLon\": 113.93263822751037,\n" +  
29        "\t\t\"approachR\": 200.0,\n" +  
30        "\t\t\"approachVel\": 20.0,\n" +  
31        "\t\t\"homeAlt\": 0.0,\n" +  
32        "\t\t\"homeAltType\": 2,\n" +  
33        "\t\t\"homeLat\": 22.51400425525294,\n" +  
34        "\t\t\"homeLon\": 113.92571262532796,\n" +  
35        "\t\t\"transAlt\": 40.0\n" +  
36        "\t},\n" +  
37        "\t\"launchInfo\": {\n" +  
38        "\t\t\"altType\": 2,\n" +  
39        "\t\t\"departureAlt\": 100.0,\n" +
```

```
40  "\\t\\t\\\"departureLat\\\": 22.514004534760666,\\n\" +
41  "\\t\\t\\\"departureLon\\\": 113.91878702314557,\\n\" +
42  "\\t\\t\\\"departureR\\\": 200.0,\\n\" +
43  "\\t\\t\\\"departureVel\\\": 20.0,\\n\" +
44  "\\t\\t\\\"launchAlt\\\": 0.0,\\n\" +
45  "\\t\\t\\\"launchLat\\\": 22.51400425525294,\\n\" +
46  "\\t\\t\\\"launchLon\\\": 113.92571262532796,\\n\" +
47  "\\t\\t\\\"transAlt\\\": 40.0\\n\" +
48  "\\t},\\n\" +
49  "\\t\\\"missionInfo\\\": [{\\n\" +
50  "\\t\\t\\\"FinishMove\\\": 1,\\n\" +
51  "\\t\\t\\\"IANum\\\": 0,\\n\" +
52  "\\t\\t\\\"InterestArea\\\": [],\\n\" +
53  "\\t\\t\\\"LinkLostMove\\\": 2,\\n\" +
54  "\\t\\t\\\"WPNum\\\": 6,\\n\" +
55  "\\t\\t\\\"subMissionInfo\\\": {\\n\" +
56  "\\t\\t\\t\\\"airLineDir\\\": 0.0,\\n\" +
57  "\\t\\t\\t\\\"baseAlt\\\": 0.0,\\n\" +
58  "\\t\\t\\t\\\"focalLength\\\": 25.6,\\n\" +
59  "\\t\\t\\t\\\"overlapAlong\\\": 0,\\n\" +
60  "\\t\\t\\t\\\"overlapSide\\\": 0,\\n\" +
61  "\\t\\t\\t\\\"pixNumX\\\": 22489,\\n\" +
62  "\\t\\t\\t\\\"pixNumY\\\": 12637,\\n\" +
63  "\\t\\t\\t\\\"resolution\\\": 0.0,\\n\" +
64  "\\t\\t\\t\\\"sensorOrient\\\": 1,\\n\" +
65  "\\t\\t\\t\\\"sensorSizeX\\\": 35.983963,\\n\" +
66  "\\t\\t\\t\\\"sensorSizeY\\\": 20.219847\\n\" +
67  "\\t\\t},\\n\" +
68  "\\t\\t\\\"subMissionType\\\": 1,\\n\" +
69  "\\t\\t\\\"wpInfo\\\": [{\\n\" +
70  "\\t\\t\\t\\\"actionParam1\\\": 2.0,\\n\" +
71  "\\t\\t\\t\\\"gimbalPitch\\\": -42.0,\\n\" +
72  "\\t\\t\\t\\\"gimbalYaw\\\": 71.0,\\n\" +
73  "\\t\\t\\t\\\"payloadAction\\\": 2,\\n\" +
74  "\\t\\t\\t\\\"wpAlt\\\": 100.0,\\n\" +
75  "\\t\\t\\t\\\"wpAltType\\\": 2,\\n\" +
76  "\\t\\t\\t\\\"wpClimbMode\\\": 1,\\n\" +
77  "\\t\\t\\t\\\"wpIndex\\\": 1,\\n\" +
78  "\\t\\t\\t\\\"wpLat\\\": 22.517180545105873,\\n\" +
79  "\\t\\t\\t\\\"wpLon\\\": 113.92888891518089,\\n\" +
80  "\\t\\t\\t\\\"wpRadius\\\": 200.0,\\n\" +
81  "\\t\\t\\t\\\"wpReserved1\\\": 0.0,\\n\" +
82  "\\t\\t\\t\\\"wpTurnMode\\\": 2,\\n\" +
83  "\\t\\t\\t\\\"wpTurnParam1\\\": 1.0,\\n\" +
84  "\\t\\t\\t\\\"wpType\\\": 4,\\n\" +
85  "\\t\\t\\t\\\"wpVel\\\": 20.0\\n\" +
86  "\\t\\t}, {\\n\" +
```

```
87      "\t\t\t\"actionParam1\": 30.0,\n" +
88      "\t\t\t\"gimbalPitch\": -31.0,\n" +
89      "\t\t\t\"gimbalYaw\": -59.0,\n" +
90      "\t\t\t\"payloadAction\": 3,\n" +
91      "\t\t\t\"wpAlt\": 100.0,\n" +
92      "\t\t\t\"wpAltType\": 2,\n" +
93      "\t\t\t\"wpClimbMode\": 1,\n" +
94      "\t\t\t\"wpIndex\": 2,\n" +
95      "\t\t\t\"wpLat\": 22.51922217962033,\n" +
96      "\t\t\t\"wpLon\": 113.92471539799595,\n" +
97      "\t\t\t\"wpRadius\": 200.0,\n" +
98      "\t\t\t\"wpReserved1\": 0.0,\n" +
99      "\t\t\t\"wpTurnMode\": 2,\n" +
100     "\t\t\t\"wpTurnParam1\": 1.0,\n" +
101     "\t\t\t\"wpType\": 4,\n" +
102     "\t\t\t\"wpVel\": 20.0\n" +
103     "\t\t}, {\n" +
104     "\t\t\t\"actionParam1\": 0.0,\n" +
105     "\t\t\t\"gimbalPitch\": -38.0,\n" +
106     "\t\t\t\"gimbalYaw\": 96.0,\n" +
107     "\t\t\t\"payloadAction\": 4,\n" +
108     "\t\t\t\"wpAlt\": 100.0,\n" +
109     "\t\t\t\"wpAltType\": 2,\n" +
110     "\t\t\t\"wpClimbMode\": 1,\n" +
111     "\t\t\t\"wpIndex\": 3,\n" +
112     "\t\t\t\"wpLat\": 22.519885273960526,\n" +
113     "\t\t\t\"wpLon\": 113.93204621923553,\n" +
114     "\t\t\t\"wpRadius\": 200.0,\n" +
115     "\t\t\t\"wpReserved1\": 0.0,\n" +
116     "\t\t\t\"wpTurnMode\": 2,\n" +
117     "\t\t\t\"wpTurnParam1\": 1.0,\n" +
118     "\t\t\t\"wpType\": 4,\n" +
119     "\t\t\t\"wpVel\": 20.0\n" +
120     "\t\t}, {\n" +
121     "\t\t\t\"actionParam1\": 0.0,\n" +
122     "\t\t\t\"gimbalPitch\": -42.0,\n" +
123     "\t\t\t\"gimbalYaw\": 20.0,\n" +
124     "\t\t\t\"payloadAction\": 0,\n" +
125     "\t\t\t\"wpAlt\": 100.0,\n" +
126     "\t\t\t\"wpAltType\": 2,\n" +
127     "\t\t\t\"wpClimbMode\": 1,\n" +
128     "\t\t\t\"wpIndex\": 4,\n" +
129     "\t\t\t\"wpLat\": 22.516633741529645,\n" +
130     "\t\t\t\"wpLon\": 113.93664236660521,\n" +
131     "\t\t\t\"wpRadius\": 200.0,\n" +
132     "\t\t\t\"wpReserved1\": 0.0,\n" +
133     "\t\t\t\"wpTurnMode\": 2,\n" +
```

```

134     "\t\t\t\"wpTurnParam1\": 1.0,\n" +
135     "\t\t\t\"wpType\": 4,\n" +
136     "\t\t\t\"wpVel\": 20.0\n" +
137     "\t\t}, {\n" +
138     "\t\t\t\"actionParam1\": 0.0,\n" +
139     "\t\t\t\"gimbalPitch\": -42.0,\n" +
140     "\t\t\t\"gimbalYaw\": 20.0,\n" +
141     "\t\t\t\"payloadAction\": 0,\n" +
142     "\t\t\t\"wpAlt\": 100.0,\n" +
143     "\t\t\t\"wpAltType\": 2,\n" +
144     "\t\t\t\"wpClimbMode\": 1,\n" +
145     "\t\t\t\"wpIndex\": 5,\n" +
146     "\t\t\t\"wpLat\": 22.513700066361963,\n" +
147     "\t\t\t\"wpLon\": 113.9360415517898,\n" +
148     "\t\t\t\"wpRadius\": 200.0,\n" +
149     "\t\t\t\"wpReserved1\": 0.0,\n" +
150     "\t\t\t\"wpTurnMode\": 4,\n" +
151     "\t\t\t\"wpTurnParam1\": 2.0,\n" +
152     "\t\t\t\"wpType\": 4,\n" +
153     "\t\t\t\"wpVel\": 20.0\n" +
154     "\t\t}, {\n" +
155     "\t\t\t\"actionParam1\": 0.0,\n" +
156     "\t\t\t\"gimbalPitch\": -42.0,\n" +
157     "\t\t\t\"gimbalYaw\": 20.0,\n" +
158     "\t\t\t\"payloadAction\": 0,\n" +
159     "\t\t\t\"wpAlt\": 100.0,\n" +
160     "\t\t\t\"wpAltType\": 2,\n" +
161     "\t\t\t\"wpClimbMode\": 1,\n" +
162     "\t\t\t\"wpIndex\": 6,\n" +
163     "\t\t\t\"wpLat\": 22.512379932020906,\n" +
164     "\t\t\t\"wpLon\": 113.94115085257494,\n" +
165     "\t\t\t\"wpRadius\": 200.0,\n" +
166     "\t\t\t\"wpReserved1\": 0.0,\n" +
167     "\t\t\t\"wpTurnMode\": 3,\n" +
168     "\t\t\t\"wpTurnParam1\": 10.0,\n" +
169     "\t\t\t\"wpType\": 4,\n" +
170     "\t\t\t\"wpVel\": 20.0\n" +
171     "\t\t}]]\n" +
172     "\t}}]\n" +
173     "}}";
174 PathPlanningParameter pathPlanningParameter = new
Gson().fromJson(waypointMission,
PathPlanningParameter.class); //getPathPlanningParameter(taskModel,
droneLocation, subMissionInfo);
175
176 //返回0表示成功, 返回非0表示失败
177 int res = NativeHelper.writeNewMissionFile(filePath, pathPlanningParameter);

```

```
178 AutelLog.d("NativeHelper", " writeMissionFile result -> " + res);
179 Toast.makeText(this, "writeMissionFile result -> " + res,
    Toast.LENGTH_SHORT).show();
```

4.2.5航线任务上传、执行

1、开始上传任务文件到飞机，任务上传前先要保存任务文件到本地路径，上传成功后需要通知飞控文件更新

初始化任务MissionManager

```
1 MissionManager missionManager = product.getMissionManager();
2 //设置任务执行时实时任务数据状态监听
3 missionManager.setRealTimeInfoListener(new CallbackWithOneParam<RealTimeInfo>
    () {
4     @Override
5     public void onSuccess(RealTimeInfo realTimeInfo) {
6         CruiserWaypointRealTimeInfoImpl info =
            (CruiserWaypointRealTimeInfoImpl) realTimeInfo;
7         AutelLog.d("MissionRunning", "timeStamp:" + info.timeStamp + ",speed:"
            + info.speed + ",isArrived:" + info.isArrived +
8             ",isDirecting:" + info.isDirecting + ",waypointSequence:" +
            info.waypointSequence + ",actionSequence:" + info.actionSequence +
9             ",photoCount:" + info.photoCount + ",MissionExecuteState:" +
            info.executeState + ",missionID:" + info.missionID);
10    }
11
12    @Override
13    public void onFailure(AutelError autelError) {
14
15    }
16 });
```

2、初始化任务航线任务对象

```
1
2 CruiserWaypointMission autelMission = new CruiserWaypointMission();
3
4 autelMission.missionId =
    BytesUtils.getInt(UUID.randomUUID().toString().replace("-", "").getBytes()); //
    任务id
5 autelMission.missionType = com.autel.common.mission.MissionType.Waypoint; //任务
    类型(Waypoint(航点)、RECTANGLE(矩形)、POLYGON(多边形))
6 autelMission.finishedAction = CruiserWaypointFinishedAction.RETURN_HOME;
```

```
7
8  if (flyMode != FlyMode.DISARM) {
9      Toast.makeText(DFWayPointActivity.this, "当前飞行模式，不能执行",
10         Toast.LENGTH_LONG).show();
11         return;
12     }
13     if (null != missionManager) {
14         autelMission.localMissionFilePath = filePath;
15         missionManager.prepareMission(autelMission, new
16             CallbackWithOneParamProgress<Boolean>() {
17             @Override
18             public void onProgress(float v) {
19                 AutelLog.d(TAG, " prepareMission onProgress " + v);
20             }
21             @Override
22             public void onSuccess(Boolean aBoolean) {
23                 //通知飞控，文件已经上传成功 返回成功后，才能执行startMission
24                 mEvoFlyController.notifyFileStatus(FileDataType.WAYPOINT,
25                     autelMission.missionId, new CallbackWithOneParam<Boolean>() {
26                     @Override
27                     public void onSuccess(Boolean aBoolean) {
28                         AutelLog.d(TAG, " notifyFileStatus onSuccess " + aBoolean);
29                         flyState = FlyState.Prepare;
30                         AutelLog.d("prepareMission success");
31                         Toast.makeText(DFWayPointActivity.this, "prepare success",
32                             Toast.LENGTH_LONG).show();
33                     }
34                     }
35                 }
36             @Override
37             public void onFailure(AutelError autelError) {
38                 AutelLog.d("notifyFileStatus onFailure");
39             }
40         });
41     }
42     @Override
43     public void onFailure(AutelError autelError) {
44         AutelLog.d("prepareMission onFailure");
45         Toast.makeText(DFWayPointActivity.this, "prepare failed",
46             Toast.LENGTH_LONG).show();
47     }
48     });
```


3、起飞前需要飞机先自检 见自检流程【4.2.6】

同时需要检查飞机电量、基站电量、遥控器电量及上报心跳中各个模块是否存在异常

```

1 //检测飞机电量
2 battery = (CruiserBattery) product.getBattery();
3 //监听飞机电池电量变化
4 battery.setBatteryStateListener(new CallbackWithOneParam<CruiserBatteryInfo>()
5 {
6     @Override
7     public void onSuccess(CruiserBatteryInfo batteryState) {
8         AutelLog.d(" batteryState uav battery-> " +
9         batteryState.getRemainingPercent());
10         isBatteryOk = batteryState.getRemainingPercent() > 15;
11     }
12 }
13 @Override
14 public void onFailure(AutelError autelError) {
15 }
16 });

```

```

1 //监听遥控器电量变化
2 PhoneBatteryManager.getInstance().addBatteryChangeListener(TAG,
3     (current, total) -> {
4         AutelLog.debug_i(TAG, "pad battery-> " + current);
5         isBatteryOk = current > 15;
6     }
7 );

```

```

1 //基站电量
2 mEvoFlyController = (CruiserFlyController) product.getFlyController();
3 mEvoFlyController.setFlyControllerInfoListener(new
4 CallbackWithOneParam<CruiserFlyControllerInfo>() {
5     @Override
6     public void onSuccess(CruiserFlyControllerInfo evoFlyControllerInfo) {
7         int baseStationBattery =
8         evoFlyControllerInfo.getBaseStationInfo().getBaseStationBattery(); //基站电量
9         //飞机模块异常告警 在接口FlyControllerStatus类中获取

```

```

9         evoFlyControllerInfo.getFlyControllerStatus()
10    }
11
12    @Override
13    public void onFailure(AutelError autelError) {
14
15    }
16    });
17
18 }

```

FlyControllerStatus接口说明

```

1  /**
2   * This interface is used to save the flight status: flight mode, main flight
   status.
3   */
4   public interface FlyControllerStatus {
5       /**
6        * Gets the main flight status.
7        *
8        * @return a MainFlyState enum.
9        */
10      MainFlyState getMainFlyState();
11
12      /**
13       * 飞行形态
14       *
15       * @return a FlightState enum.
16       */
17      FlightState getFlightState();
18
19      /**
20       * Gets the flight mode.
21       *
22       * @return a FlyMode enum.
23       */
24      FlyMode getFlyMode();
25
26      /**
27       * 获取指南针校准状态
28       * Get compass calibration status
29       */
30      CalibrateCompassStatus getCompassStatus();
31

```

```
32  /**
33   * 姿态是否异常
34   *
35   * @return true-正常, false-异常
36   */
37  boolean isAttitudeValid();
38
39  /**
40   * 机头方向无效来判断指南针状态是否有效
41   *
42   * @return true-有效, false-无效
43   */
44  boolean isCompassValid();
45
46  /**
47   * 是否超声波数据有效
48   *
49   * @return true-有效, false-无效
50   */
51  boolean isSonarValid();
52
53  /**
54   * 是否空速有效
55   *
56   * @return true-有效, false-无效
57   */
58  boolean isAirspeedValid();
59
60  /**
61   * Tells whether the GPS signal is available.
62   *
63   * @return the GPS signal is available or not.
64   */
65  boolean isGpsValid();
66
67  /**
68   * 飞机RTK异常
69   *
70   * @return true-正常, false-异常
71   */
72  boolean isRTKValid();
73
74  /**
75   * 遥控器是否连接
76   *
77   * @return true-连接, false-未连接
78   */
79  boolean isRcConnected();
```

```

79  /**
80   * 飞机是否能起飞
81   *
82   * @return true-能起飞, false-不能起飞
83   */
84  boolean isCanTakeOff();
85
86  /**
87   * Tells whether the aircraft reaches the max altitude.
88   *
89   * @return the aircraft reaches the max altitude or not.
90   */
91  boolean isReachMaxHeight();
92
93
94  /**
95   * Tells whether the aircraft reaches the max distance.
96   *
97   * @return the aircraft reaches the max distance or not.
98   */
99  boolean isReachMaxRange();
100
101  /**
102   * 飞控温度过高
103   *
104   * @return true-过高, false-正常
105   */
106  boolean isFCHot();
107
108  /**
109   * 电池温度过高
110   *
111   * @return true-过高, false-正常
112   */
113  boolean isBatteryHot();
114
115  /**
116   * 获取电池警报错误码
117   * Get battery warning
118   */
119  BatteryWarning getBatteryWarning();
120
121  /**
122   * 获取禁飞区警报错误码
123   * Get no fly zone error code
124   * @return NoFlyZoneWarning
125   */

```

```
126     int getAirportWarning();
127
128     /**
129      * 获取飞行器自检状态
130      * @return SafeCheck
131      */
132     SafeCheck getSafeCheck();
133
134     /**
135      * IMU0是否有效
136      * @return true-有效, false-无效
137      */
138     boolean isIMU0Valid();
139
140     /**
141      * IMU1是否有效
142      * @return true-有效, false-无效
143      */
144     boolean isIMU1Valid();
145
146     /**
147      * 指南针0是否有效
148      * @return true-有效, false-无效
149      */
150     boolean isMagnetmeter0Valid();
151
152     /**
153      * 指南针1是否有效
154      * @return true-有效, false-无效
155      */
156     boolean isMagnetmeter1Valid();
157
158     /**
159      * 气压计0是否有效
160      * @return true-有效, false-无效
161      */
162     boolean isBarometer0Valid();
163
164     /**
165      * 气压计1是否有效
166      * @return true-有效, false-无效
167      */
168     boolean isBarometer1Valid();
169
170     /**
171      * 固件激活标识位
172      * @return true-激活, false-未激活
173      */
```

```
173     boolean isFirmwareFlagActive();
174
175     /**
176      * RTK 航向是否有效
177      * @return true-有效, false-无效
178      */
179     boolean isRtkHeadingValid();
180
181     /**
182      * CPU温度是否正常
183      *
184      * @return true-正常, false-异常
185      */
186     boolean isCpuTemperatureNormal();
187
188     /**
189      * 飞控板子温度是否正常
190      *
191      * @return true-正常, false-异常
192      */
193     boolean isFmuBoardTemperatureNormal();
194
195     /**
196      * 天线1是否连接
197      *
198      * @return true-连接, false-未连接
199      */
200     boolean isAntenna1Connected();
201
202     /**
203      * 天线2是否连接
204      *
205      * @return true-连接, false-未连接
206      */
207     boolean isAntenna2Connected();
208
209     /**
210      * 天线1是否短路
211      *
212      * @return true-正常, false-异常
213      */
214     boolean isAntenna1Normal();
215
216     /**
217      * 天线2是否短路
218      *
219      * @return true-正常, false-异常
```

```
220     */
221     boolean isAntenna2Normal();
222
223     /**
224     * 飞控SD卡是否正常
225     *
226     * @return true-正常, false-异常
227     */
228     boolean isFmuSdcardNormal();
229
230     /**
231     * Usb调试口是否正常
232     *
233     * @return true-正常, false-异常
234     */
235     boolean isUsbDebugNormal();
236
237     /**
238     * 分控风扇是否正常
239     *
240     * @return true-正常, false-异常
241     */
242     boolean isFmuFanNormal();
243
244     /**
245     * 是否需要弹出自检框
246     *
247     * @return true-需要, false-不需要
248     */
249     boolean isNeedChecking();
250
251     /**
252     * 固件是否匹配
253     *
254     * @return true-匹配, false-不匹配
255     */
256     boolean isModelMatch();
257
258     /**
259     * 左侧机翼是否正常
260     *
261     * @return true-正常, false-异常
262     */
263     boolean isLeftWingNormal();
264
265     /**
266     * 右侧机翼是否正常
```

```

267      *
268      * @return true-正常, false-异常
269      */
270      boolean isRightWingNormal();
271
272      /**
273       * 前向感知系统 (雷达) 是否正常
274       *
275       * @return true-正常, false-异常
276       */
277      boolean isForwardSensingSystemNormal();
278
279      /**
280       * 固定翼自动飞行高度过低
281       *
282       * @return true-正常, false-异常
283       */
284      boolean isFixedWingLowHeight();
285
286      /**
287       * 固定翼自动飞行高度是否正常
288       *
289       * @return true-正常, false-异常
290       */
291      boolean isNavigationAttitudeNormal();
292
293      /**
294       * 升降舵状态
295       *
296       * @return true-正常, false-异常
297       */
298      boolean isElevatorStateNormal();
299
300      /**
301       * 电机电调状态
302       *
303       * @return true-正常, false-异常
304       */
305      boolean isEscStateNormal();
306
307      /**
308       * 说明: 在使用移动基站模式下可用, StationType = StationType.MDBS
309       * 移动基站RTK信息是否有效
310       *
311       * @return true-有效, false-无效
312       */
313      boolean isBaseStationRTKValid();

```



```

314
315 /**
316  * 说明：在使用移动基站模式下可用,StationType = StationType.MDBS
317  * 移动基站RTK Heading2 信息是否有效
318  *
319  * @return true-有效, false-无效
320  */
321 boolean isBaseStationRTKHeadingTwoValid();
322
323 /**
324  * 说明：在使用移动基站模式下可用,StationType = StationType.MDBS
325  * 移动基站RTK Heading 信息是否有效
326  *
327  * @return true-有效, false-无效
328  */
329 boolean isBaseStationRTKHeadingValid();
330
331 /**
332  * 说明：在使用移动基站模式下可用,StationType = StationType.MDBS
333  * 移动基站RTK 位置信息是否有效
334  *
335  * @return true-有效, false-无效
336  */
337 boolean isBaseStationRTKPositionValid();
338
339 /**
340  * 说明：在使用移动基站模式下可用,StationType = StationType.MDBS
341  * 移动基站RTK 速度信息是否有效
342  *
343  * @return true-有效, false-无效
344  */
345 boolean isBaseStationRTKSpeedValid();
346
347 /**
348  * 说明：在使用移动基站模式下可用,StationType = StationType.MDBS
349  * 移动基站RTK 姿态信息是否正常
350  *
351  * @return true-正常, false-异常
352  */
353 boolean isBaseStationRTKAttitudeNormal();
354
355 /**
356  * 说明：在使用移动基站模式下可用,StationType = StationType.MDBS
357  * 移动返航点信息是否正常
358  *
359  * @return true-正常, false-异常
360  */

```

```
361     boolean isMoveHomePositionNormal();
362
363     /**
364      * 说明：在使用移动基站模式下可用, StationType = StationType.MDBS
365      * 移动基返航点精度差是否正常
366      *
367      * @return true-正常, false-异常
368      */
369     boolean isMoveHomePrecisionNormal();
370
371     /**
372      * 是否需要同步电子围栏信息
373      *
374      * @return true-不需要, false-需要同步
375      */
376     boolean isNeedUploadGenFence();
377
378     /**
379      * 当前是由主设备还是从设备控制飞机
380      *
381      * @return false-从设备 true-主设备
382      */
383     boolean isMainDevice();
384
385     /**
386      * 是否处于受限制的国家禁飞区内
387      *
388      * @return false-不处于受限制的国家禁飞区内 true-处于受限制的国家禁飞区内
389      */
390     boolean isInRestrictedCountryNoFlyZone();
391
392     /**
393      * 低电量返航提示标志位
394      *
395      * @return false-正常 true-异常
396      */
397     boolean isLowBatteryReturnHomePromptFlag();
398
399     /**
400      * 升降舵通讯标志位
401      *
402      * @return true-正常, false-异常
403      */
404     boolean isElevatorStateCommunicationNormal();
405
406     /**
407      * GPS通讯标志位
```

```

408      *
409      * @return true-正常, false-异常
410      */
411      boolean isGpsCommunicationNormal();
412
413      /**
414       * RTK通讯标志位
415       *
416       * @return true-正常, false-异常
417       */
418      boolean isRTKCommunicationNormal();
419
420      /**
421       * 执行器通讯故障
422       *
423       * @return true-正常, false-异常
424       */
425      boolean isActuatorCommunicationNormal();
426
427      /**
428       * 左机翼折叠机构状态
429       * @return true-正常, false-异常
430       */
431      boolean isLeftFoldMechanismNormal();
432
433      /**
434       * 右机翼折叠机构状态
435       * @return true-正常, false-异常
436       */
437      boolean isRightFoldMechanismNormal();
438
439      /**
440       * 折叠机构状态
441       * @return FoldingMechanismStatus
442       */
443      FoldingMechanismStatus getFoldingMechanismStatus();
444
445      /**
446       * 机翼状态
447       * @return FoldingMechanismStatus
448       */
449      AllWingStatus getAllWingStatus();
450
451      /**
452       * 导航位置信息
453       *
454       * @return true-正常, false-异常

```

```
455     */
456     boolean isNavigationPositionNormal();
457
458     /**
459     * 磁航向与RTK航向是否吻合
460     * @return true-吻合, false-不吻合
461     */
462     boolean isNavigationAndRTKNormal();
463
464     /**
465     * 磁力计通讯异常
466     * @return true-正常, false-异常
467     */
468     boolean isMagnetometerCommunicationNormal();
469
470     /**
471     * 磁力计参数更新失败
472     * @return true-正常, false-异常
473     */
474     boolean isMagnetometerParameterFailed();
475
476     //结合flightMode 判断目标是否属于打击范围内
477     /**
478     * 是否在打击范围内
479     * @return true-在打击范围内, false-不在打击范围内
480     */
481     boolean isTargetATTKRange();
482
483     /**
484     * 左机翼动力机型与机身是否匹配
485     * @return true-匹配, false-不匹配
486     */
487     boolean isLeftWingPowerModelMatch();
488
489
490     /**
491     * 右机翼动力机型与机身是否匹配
492     * @return true-匹配, false-不匹配
493     */
494     boolean isRightWingPowerModelMatch();
495
496     /**
497     * 飞机是否开始降落, 可控制机巢打开舱门, 适用于机巢
498     * @return true-开始降落, false-未开始降落
499     */
500     boolean isLanding();
501
```

```
502 }
```

4、任务文件通知飞控成功后，调用开始执行任务指令

```
1  if (null != missionManager) {
2      missionManager.startMission(new CallbackWithOneParam<Pair<Boolean,
    FlightErrorState>>() {
3          @Override
4          public void onSuccess(Pair<Boolean, FlightErrorState>
    booleanFlightErrorStatePair) {
5              flyState = FlyState.Start;
6              //booleanFlightErrorStatePair.first == true 说明可以安全起飞
7              Toast.makeText(DFWayPointActivity.this, "start result " +
    booleanFlightErrorStatePair.first, Toast.LENGTH_LONG).show();
8          }
9
10         @Override
11         public void onFailure(AutelError autelError) {
12
13         }
14     });
15
16 }
```

龙鱼航线任务不支持暂停、恢复等功能，下载任务可以根据实时任务监听返回的missionID来和本地保存的任务MissionID来做恢复。

4.2.6 自检流程

1、概述：任务起飞前需要先给飞机自检，确认飞机各个模块均正常后，才能上传航线任务文件给飞机执行。

2、开始自检指令消息

```
1
2  CruiserFlyController mEvoFlyController = (CruiserFlyController)
    product.getFlyController();
3  if (null != mEvoFlyController) {
4      new IOUirunnable<Boolean>() {
5          @Override
6          protected Observable<Boolean> generateObservable() {
7              return mEvoFlyController.toRx().autoSafeCheck(modelType);
8          }
9      }
```

```

10         @Override
11         public void onNext(@NonNull final Boolean success) {
12             super.onNext(success);
13
14
15         }
16
17         @Override
18         public void onError(@NonNull Throwable e) {
19             super.onError(e);
20
21         }
22     }.execute();
23 }

```

自检的状态可以通过飞控上报的心跳消息中查询到，收到心跳消息 飞机校准完成后，去查询校准结果

```

1 mEvoFlyController = (CruiserFlyController) product.getFlyController();
2 mEvoFlyController.setFlyControllerInfoListener(new
  CallbackWithOneParam<CruiserFlyControllerInfo>() {
3
4     @Override
5     public void onSuccess(CruiserFlyControllerInfo evoFlyControllerInfo) {
6         AutelLog.d(TAG, "setFlyControllerInfoListener flyMode->" + flyMode);
7         if (null != evoFlyControllerInfo && null !=
  evoFlyControllerInfo.getFlyControllerStatus()) {
8             SafeCheck safeCheck =
  evoFlyControllerInfo.getFlyControllerStatus().getSafeCheck();
9             //飞行器自检完成
10            if (safeCheck == SafeCheck.COMPLETE) {
11                if (!isDroneCheckFinish) {
12                    isDroneCheckFinish = true;
13                    //查询自检结果
14                    getAutoCheckResult(ModelType.ALL);
15                }
16            }
17        }
18    }
19 }
20
21 @Override
22 public void onFailure(AutelError autelError) {
23
24 }
25 });

```

3、查询校准结果

```
1 private void getAutoCheckResult(ModelType modelType) {
2     if (null != mEvoFlyController) {
3         new IOUiRunnable<AutoSafeState>() {
4             int retryCount = 0;
5
6             @Override
7             protected Observable<AutoSafeState> generateObservable() {
8                 return
9                 mEvoFlyController.toRx().getAutoSafeCheck(modelType);
10            }
11
12            @Override
13            public void onNext(@NonNull final AutoSafeState safeState) {
14                super.onNext(safeState);
15                isDroneOk = safeState.isLeftSteerNormal()
16                    && safeState.isRightSteerNormal()
17                    && safeState.isBehindSteerNormal()
18                    && safeState.isAirSpeedNormal()
19                    && safeState.isImu1Normal()
20                    && safeState.isImu2Normal()
21                    && (safeState.isGPSNormal() ||
22                    safeState.isRTKNormal())
23                    && safeState.isMagnetometer1normal()
24                    && safeState.isMagnetometer2normal()
25                    && safeState.isUltrasonicNormal()
26                    && safeState.isBarometerNormal()
27                    && safeState.isBatteryNormal()
28                    && safeState.isGimbalNormal()
29                    && safeState.isRemoteControllerNormal(); //900M连接异常
30
31                AutelLog.debug_i(TAG, "showAutoCheckResult
32                isBehindSteerNormal:" + safeState.isBehindSteerNormal()
33                + " isLeftSteerNormal " +
34                safeState.isLeftSteerNormal()
35                + " isRightSteerNormal " +
36                safeState.isRightSteerNormal()
37                + " isFontMotorNormal " +
38                safeState.isFontMotorNormal()
39                + " isLeftMotorNormal " +
40                safeState.isLeftMotorNormal()
41                + " isRightMotorNormal " +
42                safeState.isRightMotorNormal()
```

```

34         + " isAirSpeedNormal " +
safeState.isAirSpeedNormal()
35         + " isBarometerNormal " +
safeState.isBarometerNormal()
36         + " isBatteryNormal " + safeState.isBatteryNormal()
37         + " isGimbalNormal " + safeState.isGimbalNormal()
38         + " isGPSNormal " + safeState.isGPSNormal()
39         + " isRTKNormal " + safeState.isRTKNormal()
40         + " isMagnetometer1normal " +
safeState.isMagnetometer1normal()
41         + " isMagnetometer2normal " +
safeState.isMagnetometer2normal()
42         + " isImu1Normal " + safeState.isImu1Normal()
43         + " isImu2Normal " + safeState.isImu2Normal()
44         + " isUltrasonicNormal " +
safeState.isUltrasonicNormal()
45         + " isGPSNormal " + safeState.isGPSNormal()
46         + " isRTKNormal " + safeState.isRTKNormal()
47 //         + " isPayloadNormal " + safeState.isPayloadNormal()
48         + " isRemoteControllerNormal " +
safeState.isRemoteControllerNormal()
49 //         + " flightMode " +
mRequestManager.getApplicationState().getFlightType()//需要先将遥控器档位拨至A档
50         + " isDroneOk " + isDroneOk);
51         Toast.makeText(getApplicationContext(), "autoCheck finish "
+ isDroneOk, Toast.LENGTH_LONG).show();
52
53     }
54
55     @Override
56     public void onError(@NonNull Throwable e) {
57         super.onError(e);
58         retryCount++;
59         if (retryCount < 3) {
60             getAutoCheckResult(modelType);
61         } else {
62
63         }
64     }
65     }.execute();
66 }
67 }

```

自检结果AutoSafeState接口说明


```
1  /**
2   * 前电机
3   *
4   * @return true-正常, false-异常
5   */
6  boolean isFontMotorNormal();
7
8  /**
9   * 后电机
10  *
11  * @return true-正常, false-异常
12  */
13 boolean isBehindMotorNormal();
14
15 /**
16  * 左电机
17  *
18  * @return true-正常, false-异常
19  */
20 boolean isLeftMotorNormal();
21
22 /**
23  * 右电机
24  *
25  * @return true-正常, false-异常
26  */
27 boolean isRightMotorNormal();
28
29 /**
30  * 左舵机
31  *
32  * @return true-正常, false-异常
33  */
34 boolean isLeftSteerNormal();
35
36 /**
37  * 右舵机
38  *
39  * @return true-正常, false-异常
40  */
41 boolean isRightSteerNormal();
42
43 /**
44  * 后舵机
45  *
46  * @return true-正常, false-异常
47  */
```

```
48 boolean isBehindSteerNormal();
49
50 /**
51  * 空速计
52  *
53  * @return true-正常, false-异常
54  */
55 boolean isAirSpeedNormal();
56
57 /**
58  * 超声
59  *
60  * @return true-正常, false-异常
61  */
62 boolean isUltrasonicNormal();
63
64 /**
65  * RTK
66  *
67  * @return true-正常, false-异常
68  */
69 boolean isRTKNormal();
70
71 /**
72  * GPS
73  *
74  * @return true-正常, false-异常
75  */
76 boolean isGPSNormal();
77
78 /**
79  * MAG (磁力计)
80  *
81  * @return true-正常, false-异常
82  */
83 boolean isMagnetometer1Normal();
84
85 /**
86  * IMU
87  *
88  * @return true-正常, false-异常
89  */
90 boolean isImu1Normal();
91
92 /**
93  * 气压计
94  *
```

```
95  * @return true-正常, false-异常
96  */
97  boolean isBarometerNormal();
98
99  /**
100  * RC
101  *
102  * @return true-正常, false-异常
103  */
104  boolean isRemoteControllerNormal();
105
106  /**
107  * 负载
108  *
109  * @return true-正常, false-异常
110  */
111  boolean isPayloadNormal();
112
113  /**
114  * MAG2 (磁力计)
115  *
116  * @return true-正常, false-异常
117  */
118  boolean isMagnetometer2normal();
119
120  /**
121  * IMU2
122  *
123  * @return true-正常, false-异常
124  */
125  boolean isImu2Normal();
126
127  /**
128  * 电池
129  *
130  * @return true-正常, false-异常
131  */
132  boolean isBatteryNormal();
133
134  /**
135  * 云台
136  *
137  * @return true-正常, false-异常
138  */
139  boolean isGimbalNormal();
140
141  /**
```

```

142 * 高度是否正常
143 * @return true-正常, false-异常
144 */
145 boolean isAttitudeNormal();
146
147 /**
148 * RTK 航向是否正常
149 * @return true-正常, false-异常
150 */
151 boolean isRTKHeadingNormal();
152 /**
153 * 导航姿态
154 * @return true-正常, false-异常
155 */
156 boolean isNavigationAttitudeNormal();
157
158 /**
159 * 保养提醒
160 *
161 * @return true-正常, false-异常
162 */
163 boolean isMaintenancReminder();
164
165 /**
166 * 版本匹配
167 *
168 * @return true-匹配, false-不匹配
169 */
170 boolean isVersionMatching();
171
172 /**
173 * 版本匹配
174 *
175 * @return true-匹配, false-不匹配
176 */
177 boolean isElectronicFenceNormal();
178 /**
179 * 磁航向与RTK航向是否吻合
180 * @return true-吻合, false-不吻合
181 */
182 boolean isNavigationAndRTKNormal();

```

4.2.7电子围栏文件

1、写入围栏文件到本地

```
1 /**
2  * 写入地理围栏文件
3  * @param filePath 文件路径
4  * @param data 地理围栏数据
5  * @return 是否写入成功
6  */
7 public static native boolean writeGeoFenceFile(String filePath,
    ArrayList<GeoFenceModel> data);
```

2、上传电子围栏文件前先检查此文件是否存在

```
1 ((CruiserFlyController) flyController).checkFileData(FileDataType.GEOFENCE, new
    File(filePath), new CallbackWithOneParam<Boolean>() {
2
3     @Override
4     public void onSuccess(Boolean success) {
5         if(success){
6             //上传围栏文件
7         }
8         AutelLog.debug_i(TAG, "checkGeoFenceFile->onSuccess, success: "
        + success + "-" + id);
9     }
10
11     @Override
12     public void onFailure(AutelError autelError) {
13         if (autelError == AutelError.FLY_UPLOAD_NONSUPPORT_FILE_TYPE) {
14             //不支持的文件类型
15             //说明飞控版本不支持，飞控的版本尚未升级
16             //不需要发送文件
17             AutelLog.debug_i(TAG, "checkGeoFenceFile->onFailure,
            autelError: " + autelError.getDescription() + ", fly-control not support
            GEOFENCE, destroy thread, filePath: " + filePath + ", id: " + id);
18
19         } else if (mRetryCount4CheckGeoFenceFile++ > MAX_RETRY_COUNT) {
20             AutelLog.debug_i(TAG, "checkGeoFenceFile->onFailure,
            autelError: " + autelError.getDescription() + ", failure " + MAX_RETRY_COUNT +
            "times, no retry, and destroy thread, filePath: " + filePath + ", id: " + id);
21
22         }
23     }
24 });
```

3、上传围栏文件

```
1 ((CruiserFlyController) flyController).uploadFileDataByTcp(filePath, new
  CallbackWithOneParamProgress<Float>() {
2     @Override
3     public void onProgress(float progress) {
4         AutelLog.e(TAG, "uploadGeoFenceFile->uploadFileDataByTcp onProgress,
  progress: " + progress);
5     }
6
7     @Override
8     public void onSuccess(Float aFloat) {
9         //文件上传成功, 通知飞控更新文件
10        AutelLog.debug_i(TAG, "uploadGeoFenceFile->uploadFileDataByTcp
  onSuccess, aFloat: " + aFloat + "-" + id + " upload success");
11    }
12
13    @Override
14    public void onFailure(AutelError autelError) {
15        AutelLog.debug_i(TAG, "uploadGeoFenceFile->uploadFileDataByTcp
  onFailure, autelError: " + autelError.getDescription() + ", filePath: " +
  filePath + ", id: " + id);
16        sendRetryUploadFile(filePath, id);
17    }
18 });
```

4、通知飞控文件上传成功更新状态

```
1 ((CruiserFlyController) flyController).notifyFileStatus(FileDataType.GEOFENCE,
  id, new CallbackWithOneParam<Boolean>() {
2
3     @Override
4     public void onSuccess(Boolean aFloat) {
5         handleUploadSuccess();
6         AutelLog.debug_i(TAG, "notifyUploadGeoFenceStatus->onSuccess,
  aFloat: " + aFloat + ", " + needForceUpload + ", mLatestCoordinate3D: " +
  mLatestCoordinate3D + ", id-" + id + "notify completed");
7     }
8
9     @Override
10    public void onFailure(AutelError autelError) {
11        if (mRetryCount4NotifyGeoFenceFile++ > MAX_RETRY_COUNT) {
12            //需要重试上传
```

```

13         AutelLog.debug_i(TAG, "notifyUploadGeoFenceStatus-
>onFailure, autelError: " + autelError.getDescription() + ", failure " +
MAX_RETRY_COUNT + "times, no retry, and destroy thread, id: " + id);
14     }
15     AutelLog.debug_i(TAG, "notifyUploadGeoFenceStatus->onFailure,
autelError: " + autelError.getDescription() + ", id: " + id);
16 }
17 });

```

5、相册功能实现教程

1、获取相册信息

获取到AutelAlbum之后，调用对应的接口方法，拿到相册的相关数据，参考类[AlbumActivity.java](#)例如：

```

1 private List<MediaInfo> mediaItems;
2 //获取相册列表
3 product.getAlbum().getMedia(0, 10,
4     new CallbackWithOneParam<List<MediaInfo>>() {
5         @Override
6         public void onFailure(AutelError autelError) {
7             //获取失败
8         }
9
10        @Override
11        public void onSuccess(List<MediaInfo> mediaInfos) {
12            //获取成功
13            mediaItems = data;
14            //存储对应的集合 用于后面删除操作
15        }
16    });

```

其中MediaInfo为单个媒体详细信息（文档末尾有详细说明）

2、相册API接口介绍

2.1.获取相册列表

包含SD及机载闪存中获取接口

```

2  /**
3   * Gets the media data from the camera.
4   *
5   * @param albumType
6   * @param start the first index of the media list.
7   * @param count the number of elements in the media list, the max number is
8   * 500.
9   * @param callback the callback when get the media data from the camera server.
10  */
11 void getMedia(AlbumType albumType, int start, int count,
12 CallbackWithOneParam<List<MediaInfo>> callback);
13
14 /**
15  * Gets the media data from the camera server.
16  *
17  * @param start the first index of the media list.
18  * @param count the number of elements in the media list, the max number is
19  * 500.
20  * @param callback the callback when get the media data from the camera server.
21  */
22 void getMedia(int start, int count, CallbackWithOneParam<List<MediaInfo>>
23 callback);
24
25 /**
26  * Deletes the specified media list.
27  *
28  * @param deleteMediaList the specified media list which is deleted by the
29  * camera, the max number is 50.
30  * @param callback the callback when delete the specified media data from the
31  * camera.
32  */
33 void deleteMedia(List<MediaInfo> deleteMediaList,
34 CallbackWithOneParam<List<MediaInfo>> callback);
35
36 /**
37  * Deletes the specified media data.
38  * @param deleteMedia the specified media data which is used to be deleted by
39  * the camera.
40  * @param callback the callback when delete media data from the camera.
41  */
42 void deleteMedia(MediaInfo deleteMedia, CallbackWithOneParam<List<MediaInfo>>
43 callback);
44
45 /**
46  * Gets the video resolutions contained in the http header.

```



```

39  * @param mediaInfo the media information which is used to find the resolution
    ratio.
40  * @param callback the callback when get the video resolution ratio from the
    camera.
41  */
42  void getVideoResolutionFromHttpHeader(MediaInfo mediaInfo,
    CallbackWithOneParam<VideoResolutionAndFps> callback);
43
44  /**
45   * Returns the video resolutions and associated frames per second (Fps)
    contained in the local file.
46
47   * Unable to get the resolution ratio if the file does not exist or not
    generated by the aircraft camera.
48
49   * @param file the video file containing the resolution ratio.
50   */
51  VideoResolutionAndFps getVideoResolutionFromLocalFile(File file);
52
53  /**
54   * Gets the manager of the album parameter range.
55   * @return the manager of the album parameter range.
56   */
57  AlbumParameterRangeManager getParameterRangeManager();
58
59  //=====FMC =====
60
61
62  /**
63   * Returns the media data from the camera server.
64   *
65   * @param albumType 相册类型 0:全部 1:照片 2:视频
66   * @param start the first index of the media list.
67   * @param count the number of elements in the media list, the max number is
    500.
68   * @param callback the callback when get the media data from the camera server.
69   */
70  void getFMCMedia(AlbumType albumType, int start, int count,
    CallbackWithOneParam<List<MediaInfo>> callback);
71  /**
72   * Returns the media data from the camera server.
73   *
74   * @param start the first index of the media list.
75   * @param count the number of elements in the media list, the max number is
    500.
76   * @param callback the callback when get the media data from the camera server.
77   */

```

```

78 void getFMCMedia(int start, int count, CallbackWithOneParam<List<MediaInfo>>
    callback);
79
80
81 /**
82  * Returns the specified media list which is deleted by the camera server.
83  *
84  * @param deleteMediaList the specified media list which is deleted by the
    camera server, the max number is 50.
85  * @param callback the callback when delete the specified media data from the
    camera server.
86  */
87 void deleteFMCMedia(List<MediaInfo> deleteMediaList,
    CallbackWithOneParam<List<MediaInfo>> callback);
88
89 /**
90  * Returns the specified media data which is deleted by the camera server.
91  * @param deleteMedia the specified media data which is used to be deleted by
    the camera server.
92  * @param callback the callback when delete media data from the camera server.
93  */
94 void deleteFMCMedia(MediaInfo deleteMedia,
    CallbackWithOneParam<List<MediaInfo>> callback);
95
96 /**
97  * Returns the video resolutions contained in the http header.
98  * @param mediaInfo the media information which is used to find the resolution
    ratio.
99  * @param callback the callback when get the video resolution ratio from the
    camera server.
100 */
101 void getFMCVideoResolutionFromHttpHeader(MediaInfo mediaInfo,
    CallbackWithOneParam<VideoResolutionAndFps> callback);

```

2.2. 下载相册数据

1. 下载视频或者照片文件

通过AutelAlbum接口中的获取到媒体数据地址，可以将无人机中的相册（图片）下载到指定的路径，示例如下：

下载照片也一样，修改地址为照片路径就好

```

1 private int mediaStorageType = 2; // 1:SD卡 2:机载闪存卡
2 private void downloadVideo() {
3     if (null == okHttpManager) {

```

```

4      okhttpManager = new OkHttpClient.Builder().build();
5  }
6  if (null != media2Download) {
7      String videoPath = media2Download.getOriginalMedia();
8      String videoName = "Album.video";
9      if (!isEmpty(videoPath)) {
10         videoName = videoPath.substring(videoPath.lastIndexOf("/") + 1,
videoPath.length());
11     }
12     if (mediaStorageType == 2) {
13         if (videoPath.contains("/DCIM/")) {
14             videoPath = videoPath.split("/DCIM/")[0] + "/emmc" + "/DCIM/"
+ videoPath.split("/DCIM/")[1];
15         }
16     }
17     logOut("downloadVideo path " + videoPath);
18     AutelLog.d("downloadVideo path " + videoPath+" videoName->" + videoName);
19     okhttpManager.download(media2Download.getLargeThumbnail(),
Environment.getExternalStorageDirectory().getPath() + "/album/" + videoName,
new ResponseCallBack<File>() {
20         @Override
21         public void onSuccess(File file) {
22             initLocalFileList();
23             logOut("file " + file.getPath());
24         }
25
26         @Override
27         public void onFailure(Throwable throwable) {
28             logOut("download onFailure " + throwable.getMessage());
29         }
30     });
31 } else {
32     logOut("video to be download is null");
33 }
34 }

```

2.删除文件

删除文件可以从SD卡中或者机载闪存中删除，分别调用对应的接口即可，示例

```

1 mController.deleteMedia(mediaItems, new CallbackWithOneParam<List<MediaInfo>>
() {
2     @Override
3     public void onFailure(AutelError error) {
4         logOut("deleteMedia error " + error.getDescription());
5     }

```

```

6
7     @Override
8     public void onSuccess(List<MediaInfo> data) {
9         logOut("deleteMedia size " + data.size());
10        for (MediaInfo item : data) {
11            Log.v(TAG, "deleteMedia data onFailure " +
12            item.getOriginalMedia() + " " + item.getFileSize() + " " +
13            item.getFileTimeString() + " SmallThumbnail " + item.getSmallThumbnail());
14        }
15    }
16 }
17 }
18 });

```

6、视频解码功能实现教程

1.1 播放器Api说明

使用AutelPlayerManager这个类的单实例及AutelPlayer、AutelPlayerView类的实例来调用接口。

AutelPlayerManager.java相关接口说明

```

1  /**
2   * AutelPlayerManager的单例类对象
3   */
4   public synchronized static AutelPlayerManager getInstance()
5
6
7   /**
8   * 添加AutelPlayer实例到mAutelPlayerList中
9   *
10  * @param player 要添加的AutelPlayer实例
11  */
12  public void addAutelPlayer(AutelPlayer player)
13
14
15  /**
16  * 添加MediaCodec监听器
17  * @param key 键值
18  * @param callbackWith 回调接口OnRenderFrameInfoListener
19  */
20  public void addCodecListeners(String key, final OnRenderFrameInfoListener
    callbackWith)

```

AutelPlayer.java相关接口说明

```

1 /**
2  * 构造方法
3  * @param channelId: 通道ID 参看SDKConstants定义
4  */
5 public AutelPlayer(int channelId)
6
7 /**
8  * 添加用于视频的视图
9  * @param view AutelPlayerView对象, 用于显示视频的视图
10 */
11 public void addVideoView(AutelPlayerView view)
12
13 /**
14  * 移除视频视图
15 */
16 public void removeVideoView()
17
18 /**
19  * 设置视频流监听器
20  * @param ls 实现了IVideoStreamListener接口的监听器对象
21 */
22 public void setVideoInfoListener(IVideoStreamListener ls)

```

AutelPlayerView.java相关接口说明

```

1 /**
2  * 创建一个新的AutelPlayerView视图。
3  *
4  * @param context 与该视图关联的上下文Context。
5  */
6 public AutelPlayerView(Context context)
7
8 /**
9  * 创建一个新的AutelPlayerView视图。
10 */
11 * @param context 与该视图关联的上下文Context。
12 * @param attrs 用于填充该视图的XML标签的属性。
13 */
14 public AutelPlayerView(Context context, AttributeSet attrs)

```

1.2 播放器接口调用实例

```

1 //创建播放TextureView
2 private AutelPlayerView codecView;
3 private AutelPlayer mAutelPlayer;
4 private fun createAutelCodecView(): AutelPlayerView? {
5     val codecView = AutelPlayerView(activity)
6     val params = LinearLayout.LayoutParams(
7         ViewGroup.LayoutParams.WRAP_CONTENT,
8         ViewGroup.LayoutParams.WRAP_CONTENT
9     )
10    codecView.layoutParams = params
11    return codecView
12 }
13 //初始化播放器
14 private void initVideo() {
15     AutelLog.debug_i("initCodec","initVideo codecBase ");
16     AutelPlayerManager.getInstance().init(this, false);
17     codecView = createAutelCodecView();
18     mContentLayout.addView(codecView);
19     mAutelPlayer = new AutelPlayer(0);
20     mAutelPlayer.addVideoView(codecView);
21     AutelPlayerManager.getInstance().addAutelPlayer(mAutelPlayer);
22     mAutelPlayer.startPlayer();
23 }
24 //停止播放
25 private void stopPlayer(){
26     AutelLog.debug_i("initCodec","stopPlayer codecBase ");
27     if(null != mAutelPlayer){
28         mAutelPlayer.removeVideoView();
29         AutelPlayerManager.getInstance().removeAutelPlayer(mAutelPlayer);
30         mAutelPlayer.stopPlayer();
31         mAutelPlayer.releasePlayer();
32     }
33 }

```

7、推流示例

1、概述

龙鱼推流支持rtmp及rtsp推流，支持内外网部署SAS流媒体服务器推拉流实现。

无人机拉流方式：

1.1 rtmp推流

说明：需要在内外网有搭建SAS流媒体服务器后 在遥控器APP端连接内外网的情况下 输入rtmp推流地址后，启动开始推流，电脑才可以通过VLC播放器或者其他播放器程序输入同一地址拉流

1.2 rtmp推流到流媒体服务器实现,参考代码LivePushActivity.java

```
1 private String mRtmpUrl = "rtmp://a.rtmp.youtube.com/live2/vkzh-1udz-ak4j-mscv-29s4";
2 AutelPlayer.init();
3 Observable.interval(5,
    TimeUnit.SECONDS).subscribeOn(Schedulers.io()).subscribe(aLong ->
    NetworkProxyJni.ForceKeyFrame(0));
4 ThreadUtils.runOnNonUiThread(() -> {
5     int rtmpStart = AutelPlayer.rtmpStart(mRtmpUrl);
6     AutelAudioRecorderUtils.instance_().startRecording();
7     AutelLog.d(TAG, "====>>>rtmpStart: "+rtmpStart);
8     ThreadUtils.runOnUiThread(() -> mTvMessage1.setText("rtmpStart:
    "+rtmpStart));
9 });
10
```

1.3 停止推流

```
1 public void stopLivePush(View view) {
2     ThreadUtils.runOnNonUiThread(new Runnable() {
3         @Override
4         public void run() {
5             AutelAudioRecorderUtils.instance_().stopRecording();
6             int rtmpStop = AutelPlayer.rtmpStop();
7             AutelLog.d(TAG, "====>>>rtmpStop: "+rtmpStop);
8             ThreadUtils.runOnUiThread(() -> mTvMessage1.setText("rtmpStop:
    "+rtmpStop));
9         }
10     });
11 }
```

1.4 监听推流数据

```
1
2 AutelPlayer.setOnAutelPlayerListener(new AutelPlayer.OnAutelPlayerListener(){
3
4     @Override
```

```

5     public void onPlayerRtmpVideoAttr(int videoBitrate, int videoFramerate, int
      audioBitrate) {
6         AutelLog.d(TAG, "===>>>onPlayerRtmpVideoAttr, i: "+videoBitrate+", i1:
      "+videoFramerate+", i2: "+audioBitrate);
7     }
8 });
9
10    public interface OnAutelPlayerListener{
11
12        void onPlayerRtmpVideoAttr(int videoBitrate, int videoFramerate, int
      audioBitrate);
13    }

```

2、rtsp拉流

2.1 实现

电脑直接连接基站WiFi网络，电脑安装ffplay播放器,通过ffplay输入rtsp://192.168.2.1/realtime播放或者通过电脑VLC播放器输入rtsp地址获取到视频流。

五、平板按键消息及波轮控制

1、概述

遥控器控制包含遥控器模式设置、左右波轮、变焦、返航键、拍照/录像按键 参考 RemoteControllerActivity.java示例

2、遥控操作模式设置

```

1 RemoteControllerCommandStickMode commandStickMode =
      RemoteControllerCommandStickMode.CHINA;
2 AutelSerialManager.getInstance().setCommandStickMode(commandStickMode);

```

3、获取遥控器操作模式

```

1 AutelSerialManager.getInstance().getCommandStickMode(new
      CallbackWithOneParam<RemoteControllerCommandStickMode>() {
2     @Override

```



```

3      public void onSuccess(RemoteControllerCommandStickMode
remoteControllerCommandStickMode) {
4          logOut("getRCCommandStickMode->" + remoteControllerCommandStickMode);
5      }
6
7      @Override
8      public void onFailure(AutelError autelError) {
9
10     }
11 });

```

4、遥控器按键消息上报

```

1 AutelSerialManager.getInstance().addSerialKeystrokeListener(TAG, new
AutelSerialManager.SerialKeystrokeListener() {
2     @Override
3     public void onResponse(RemoteControllerNavigateButtonEvent
remoteControllerNavigateButtonEvent) {
4         logOut("setRemoteButtonControllerListener onResponse " +
remoteControllerNavigateButtonEvent);
5
6     }
7 });

```

5、遥控器开始/停止校准

```

1 //开始校准指令
2 AutelSerialManager.getInstance().startCommand(AutelSerialManager.COMMAND_RC_STA
RT);
3 //结束校准指令
4 AutelSerialManager.getInstance().startCommand(AutelSerialManager.COMMAND_RC_END
);
5

```

6、遥控器校准数据上报

```

1 AutelSerialManager.getInstance().addSerialRcListener(TAG,
2     response -> ThreadUtils.runOnUiThread(() -> showCalibrateUi(response))
3 );

```

7、遥控器波轮数据上报

```
1 AutelSerialManager.getInstance().addSerialRcListener(TAG, new
  AutelSerialManager.SerialRcListener() {
2     @Override
3     public void onResponse(int[] ints) {
4         logOut("setRemoteButtonControllerListener onResponse " + ints[0] + " "
          + ints[1] + " " + ints[2] + " " + ints[3] + " " + ints[4] + " " + ints[5]);
5     }
6 });
```